

DeSFAM: Một Framework Thích Nghi Dựa trên eBPF và AI để Bảo Mật Container Đám Mây Theo Thời Gian Thực

SEHAR ZEHRA¹, HASSAN JAMIL SYED², FAHAD SAMAD³, VÀ UMMAY FASEEHA¹, (Thành viên, IEEE)

Tóm tắt nội dung—Các ứng dụng được đóng gói container mang lại môi trường triển khai nhẹ và có khả năng mở rộng, nhưng vẫn tiềm ẩn các rủi ro bảo mật do dùng chung nhân hệ điều hành. Chúng tôi giới thiệu DeSFAM (Dynamic eBPF-driven Syscall Filtering and Anomaly Mitigation), một framework bảo mật thời gian thực thực thi nguyên tắc đặc quyền tối thiểu trong việc sử dụng lời gọi hệ thống (syscall) và phát hiện các bất thường về hành vi. DeSFAM tích hợp: 1) lập hồ sơ syscall lai thông qua phân tích tĩnh và theo dõi eBPF động; 2) SyscallAD (Phát hiện bất thường lời gọi hệ thống), một bộ phát hiện bất thường có độ trễ thấp kết hợp Bộ mã hóa biến phân tự động (VAE) và Rừng cô lập (iForest); 3) tính điểm rủi ro theo ngữ cảnh dựa trên ảnh xạ MITRE ATT&CK và tương quan CVE; và 4) thực thi syscall thích nghi sử dụng bản đồ eBPF và các móc LSM. Đánh giá sử dụng bộ dữ liệu DongTing và các kịch bản tấn công CVE thực tế cho thấy DeSFAM đạt được độ chính xác 94%, độ thu hồi 90%, độ trễ thực thi dưới mili-giây và chi phí hiệu năng dưới 1%. DeSFAM chặn hiệu quả các cuộc tấn công leo thang đặc quyền, cố gắng thoát container và các cuộc tấn công tiềm syscall trong môi trường container hiện đại.

Index Terms—Phát hiện bất thường, container đám mây, bảo mật container, giám sát eBPF, phát hiện mối đe dọa thời gian thực, lọc lời gọi hệ thống, tăng cường seccomp.

I. GIỚI THIỆU

Công nghệ container hóa đã trở thành nền tảng của điện toán đám mây hiện đại nhờ tính hiệu quả và khả năng mở rộng. Không giống như các máy ảo truyền thống, container chia sẻ nhân hệ điều hành (OS kernel) của máy chủ, điều này làm giảm chi phí vận hành nhưng cũng đưa ra các rủi ro bảo mật nghiêm trọng thông qua lời gọi hệ thống (syscall)—giao diện giữa không gian người dùng và nhân hệ điều hành [1], [2].

¹Khoa Khoa học Máy tính, Đại học Quốc gia FAST về Khoa học Máy tính và Khoa học Nổi bật, Karachi 75030, Pakistan

²Đại học Công nghệ và Đổi mới Châu Á Thái Bình Dương (APU), Kuala Lumpur 57000, Malaysia

³Khoa An ninh mạng, Đại học Quốc gia FAST về Khoa học Máy tính và Khoa học Nổi bật, Karachi 75030, Pakistan

Tác giả liên hệ: Hassan Jamil Syed (hassan.jamil@apu.edu.my)

Nghiên cứu này được tài trợ bởi Đại học Công nghệ và Đổi mới Châu Á Thái Bình Dương (APU), Kuala Lumpur, Malaysia.

Bản gốc tiếng Anh: *IEEE Access*, tập 13, 2025, DOI: 10.1109/ACCESS.2025.3592192. Bản dịch tiếng Việt phục vụ mục đích học thuật.

Syscall mở ra một bề mặt tấn công rộng lớn, khiến chúng trở thành mục tiêu hấp dẫn cho các hành vi leo thang đặc quyền, thoát container và nhiều khai thác khác [3]. Các giải pháp hiện có như hồ sơ seccomp chỉ cung cấp mức độ bảo vệ hạn chế. Các hồ sơ tĩnh khó duy trì và có thể chặn các thao tác hợp lệ hoặc không phát hiện được các mối đe dọa tinh vi.

Để khắc phục những hạn chế này, chúng tôi đề xuất DeSFAM, Lọc lời gọi hệ thống động dựa trên eBPF và giảm thiểu bất thường. DeSFAM cung cấp lập hồ sơ syscall thời gian thực, phát hiện bất thường sâu và thực thi chính sách thích nghi. Framework này tích hợp: (i)

- 1) Lập danh sách truy cập syscall lai sử dụng phân tích tĩnh và phân tích động dựa trên eBPF;
- 2) SyscallAD, một công cụ phát hiện kết hợp Bộ mã hóa biến phân tự động (VAE) và Rừng cô lập (iForest) để xác định các chuỗi syscall bất thường; và
- 3) Quản lý chính sách thích nghi sử dụng ảnh xạ khung chiến thuật, kỹ thuật và kiến thức đối nghịch MITRE (ATT&CK), tương quan lỗ hổng và phối nhiệm chung (CVE) và các móc eBPF LSM để kiểm soát syscall theo ngữ cảnh.

DeSFAM thực thi kiểm soát syscall chi tiết bằng cách sử dụng bản đồ eBPF và các móc LSM, chặn động các syscall có nguy cơ cao (ví dụ: `execve`, `ptrace`) và cập nhật chính sách tại thời điểm chạy mà không cần khởi động lại container. Framework này kết hợp một quy trình lập hồ sơ syscall lai với SyscallAD, một bộ phát hiện bất thường có độ trễ thấp dựa trên Bộ mã hóa biến phân tự động (VAE) và Rừng cô lập. Để ưu tiên các mối đe dọa, DeSFAM tích hợp tính điểm rủi ro theo ngữ cảnh bằng cách tương quan hành vi syscall với các chiến thuật MITRE ATT&CK và các CVE đã biết. Đánh giá sử dụng bộ dữ liệu DongTing và các kịch bản CVE thực tế cho thấy độ chính xác 94%, độ thu hồi 90%, độ trễ thực thi dưới mili-giây và chi phí hiệu năng dưới 1%.

Các đóng góp chính của chúng tôi bao gồm:

- Một quy trình lập hồ sơ lai để tạo danh sách cho phép syscall.
- SyscallAD, một bộ phát hiện bất thường VAE + iForest có độ trễ thấp.
- Tính điểm rủi ro theo ngữ cảnh sử dụng MITRE ATT&CK và tương quan CVE.

- Thực thi syscall chi tiết sử dụng bản đồ eBPF và các móc LSM.
- Vòng phản hồi thời gian chạy để tự động tăng cường chính sách dựa trên các vi phạm.

Bài báo được tổ chức như sau: Mục II trình bày các kiến thức nền tảng về lọc syscall và eBPF. Mục III thảo luận các công trình liên quan. Mục IV mô tả kiến trúc của DeSFAM. Mục V trình bày kết quả đánh giá, tiếp theo là thảo luận ở Mục VI và kết luận ở Mục VII.

II. KIẾN THỨC NỀN TẢNG

Công nghệ container hóa đã thay đổi căn bản việc triển khai phần mềm đám mây hiện đại bằng cách cung cấp các môi trường thực thi nhẹ, có khả năng mở rộng và di động. Không giống như các máy ảo (VM), vốn đóng gói một hệ điều hành hoàn chỉnh cho mỗi phiên bản, các container chia sẻ nhân OS của máy chủ trong khi cung cấp các không gian người dùng cô lập thông qua không gian tên (namespaces) và nhóm điều khiển (cgroups). Kiến trúc này giảm chi phí vận hành và tăng tốc triển khai, khiến container trở nên lý tưởng cho microservices, quy trình CI/CD và khối lượng công việc linh hoạt. Tuy nhiên, mô hình chia sẻ nhân này mang lại những rủi ro bảo mật độc đáo—đặc biệt tại giao diện syscall, nơi điều phối quyền truy cập vào các chức năng cốt lõi của nhân.

A. Bảo mật lời gọi hệ thống trong môi trường chia sẻ nhân

Syscall đóng vai trò là giao diện chính giữa các ứng dụng không gian người dùng và tài nguyên nhân. Trong các hệ thống container hóa, chúng rất quan trọng đối với việc thực thi tiến trình, thao tác tệp, quản lý bộ nhớ và giao tiếp mạng. Thật không may, khả năng rộng lớn của chúng cũng khiến chúng trở thành mục tiêu hàng đầu của những kẻ tấn công. Nhân hệ điều hành cung cấp hàng trăm syscall, nhiều trong số đó là không cần thiết cho các khối lượng công việc container hóa điển hình, do đó mở rộng bề mặt tấn công [4].

Những kẻ tấn công thường khai thác các syscall bị lạm dụng hoặc có lỗ hổng để thực thi leo thang đặc quyền, thoát khỏi container hoặc can thiệp vào môi trường máy chủ. Các mối đe dọa như vậy trở nên nghiêm trọng hơn trong các triển khai đám mây đa thuê (multi-tenant), nơi nhiều container chia sẻ cùng một nhân máy chủ và do đó cùng một không gian tên syscall. Nếu không có kiểm soát syscall nghiêm ngặt, một sự xâm phạm trong một container có thể dẫn đến di chuyển ngang hoặc chiếm quyền máy chủ.

B. Các loại mối đe dọa dựa trên lời gọi hệ thống

Lời gọi hệ thống mở ra các chức năng nhân cốt lõi, khiến chúng trở thành mục tiêu hàng đầu trong môi trường container hóa. Những kẻ tấn công khai thác chúng để leo thang đặc quyền, phá vỡ cô lập hoặc lạm dụng các hồ sơ quá rộng quyền.

Leo thang đặc quyền: Những kẻ tấn công có thể khai thác các lỗ hổng nhân, khả năng Linux bị cấu hình sai

hoặc các hồ sơ seccomp không đầy đủ để đạt được đặc quyền cao hơn từ bên trong một container. Các ví dụ bao gồm việc sử dụng `setuid()`, `clone()` hoặc `execve()` theo các trình tự cụ thể để chiếm quyền thực thi luồng hoặc vượt qua các hạn chế không gian tên.

Thoát container: Các khai thác như CVE-2022-0847 [5] (Dirty Pipe) hoặc CVE-2019-5736 [6] (Docker runc) cho phép các tiến trình độc hại vượt qua ranh giới container và tương tác với máy chủ. Những điều này thường liên quan đến việc tạo ra các chuỗi syscall độc hại can thiệp vào bộ nhớ chỉ đọc hoặc thoát khỏi cô lập không gian tên.

Lạm dụng bề mặt tấn công: Nhiều container có quyền truy cập quá rộng vào các syscall do sử dụng hồ sơ seccomp chung hoặc mặc định. Sự quá rộng quyền này có thể bị các đối thủ tinh vi khai thác để kết hợp các syscall thành các cuộc tấn công nhiều giai đoạn và tàng hình.

C. Hạn chế của các phương pháp giảm thiểu truyền thống

Các cơ chế bảo mật như AppArmor, SELinux và Seccomp cung cấp lọc syscall và thực thi chính sách. Tuy nhiên, chúng thể hiện nhiều hạn chế trong môi trường đám mây hiện đại:

- **Chính sách tĩnh không linh hoạt:** Các hồ sơ tĩnh không thích nghi với các hành vi mới hoặc vectơ mới đe dọa mới, thường dẫn đến kết quả dương tính giả (chặn quá mức) hoặc kết quả âm tính giả (bỏ sót tấn công).
- **Thiếu nhận thức về chuỗi:** Các hệ thống dựa trên quy tắc hoạt động trên các lời gọi syscall riêng lẻ mà không xem xét các phụ thuộc thời gian, khiến chúng mù quáng với các cuộc tấn công nhiều bước.
- **Nút thắt cổ chai về khả năng mở rộng:** Khi mật độ container tăng, các bộ lọc tĩnh trở nên khó duy trì và thực thi hiệu quả ở quy mô lớn.
- **Chi phí cấu hình thủ công:** Các chính sách phải được viết, kiểm tra và duy trì thủ công cho từng khối lượng công việc container—một gánh nặng không thực tế trong môi trường động.

Những điểm yếu này nhấn mạnh sự cần thiết của các giải pháp bảo mật syscall động, thông minh và nhận biết ngữ cảnh có thể mở rộng theo hạ tầng hiện đại.

D. Tiềm năng của eBPF trong giám sát lời gọi hệ thống

eBPF là một công nghệ tích hợp nhân cho phép đo lường nhân an toàn, có thể lập trình cho các sự kiện nhân, bao gồm cả syscall [7]. Nó cho phép gắn kết động logic giám sát và lọc trực tiếp tại các điểm nhập syscall (ví dụ: `sys_enter_*`) với chi phí tối thiểu. Các lợi ích chính của eBPF trong lĩnh vực bảo mật syscall bao gồm:

- **Thực thi thời gian thực:** Syscall có thể bị chặn hoặc kiểm tra ngay lập tức mà không cần chuyển đổi ngữ cảnh hay khởi động lại.
- **Lọc nhận biết ngữ cảnh:** Các chính sách có thể tính đến không gian tên, ID nhóm điều khiển, các đối số hoặc các mẫu syscall.

- **Khả năng mở rộng cao:** Các bản đồ và điểm theo dõi trong nhân hiệu quả cho phép thực thi trên hàng nghìn container đồng thời.

Tính chất có thể lập trình của eBPF khiến nó trở thành nền tảng lý tưởng để triển khai bảo mật syscall chi tiết, thời gian thực phù hợp với khối lượng công việc container.

E. Phát hiện bất thường dựa trên chuỗi lời gọi hệ thống

Các mô hình học máy (ML), đặc biệt là những mô hình phân tích chuỗi syscall, cung cấp một bổ sung mạnh mẽ cho giám sát eBPF. Thay vì chỉ dựa vào các quy tắc được xác định trước, các mô hình dựa trên chuỗi có thể học các mẫu hành vi bình thường và gắn cờ các cửa sổ syscall bất thường lệch khỏi quy trình làm việc mong đợi.

Bộ mã hóa biến phân tự động (VAE) [8] và Rừng cô lập [9] (iForest) đặc biệt hiệu quả để mô hình hóa các chuỗi syscall:

- VAE tái cấu trúc hành vi bình thường từ các biểu diễn không gian tiềm ẩn nén. Lỗi tái cấu trúc cao cho thấy sự bất thường.
- Rừng cô lập xác định các ngoại lệ thống kê dựa trên mức độ dễ dàng mà một chuỗi có thể được tách biệt khỏi các mẫu bình thường.
- Kết hợp cả hai phương pháp cải thiện tính mạnh mẽ bằng cách kết hợp tính điểm bất thường dựa trên tái cấu trúc và dựa trên cây.

Để hỗ trợ huấn luyện và xác nhận, các bộ dữ liệu công khai như bộ dữ liệu DongTing [10] cung cấp các chuỗi syscall được gán nhãn là bình thường hoặc độc hại. DeSFAM sử dụng độc quyền bộ dữ liệu DongTing, tận dụng 18.966 chuỗi được gán nhãn để huấn luyện và hiệu chỉnh các mô hình phát hiện bất thường của nó.

F. Hướng tới bảo mật lời gọi hệ thống thích nghi và có thể mở rộng

Tính chất động của điều phối container đòi hỏi các cơ chế lọc syscall thích nghi với hành vi khối lượng công việc, tích hợp thông tin tình báo về mối đe dọa và mở rộng với sự can thiệp của con người ở mức tối thiểu.

DeSFAM giải quyết các nhu cầu này bằng cách kết hợp:

- Lập hồ sơ syscall tĩnh-động lai.
- Phát hiện bất thường kết hợp học sâu (VAE) và các mô hình dựa trên cây (iForest).
- Thực thi chính sách dựa trên rủi ro sử dụng các móc eBPF LSM.
- Tính điểm rủi ro dựa trên khớp CVE và MITRE ATT&CK.
- Các cơ chế tự tăng cường để tinh chỉnh hồ sơ seccomp tại thời điểm chạy.

Thiết kế tích hợp này cho phép phát hiện thời gian thực, chặn theo ngữ cảnh và giảm thiểu bề mặt syscall lâu dài, tạo tiền đề cho bảo mật container có khả năng phục hồi và mở rộng trong môi trường chia sẻ nhân.

III. CÁC CÔNG TRÌNH LIÊN QUAN

Phần này thảo luận về các phương pháp tiên tiến được phân loại theo lọc, phát hiện bất thường, theo dõi xuất xứ và tăng cường bảo mật kernel, làm nổi bật điểm mạnh và hạn chế của chúng so với khung DeSFAM. Bảng I tóm tắt các phương pháp lọc lời gọi hệ thống và phát hiện bất thường gần đây, làm nổi bật trọng tâm vấn đề, hạn chế và kỹ thuật cốt lõi của từng phương pháp.

A. Các Phương Pháp Lọc Lời Gọi Hệ Thống

Confine [11] sử dụng phân tích tĩnh để xây dựng các hồ sơ seccomp hạn chế, đạt tỷ lệ giảm lời gọi hệ thống (SRR) là 47,6%. Tuy nhiên, tính chất tĩnh của nó ngăn cản sự thích nghi với các hành vi thời gian chạy, làm giảm hiệu quả trước các khối lượng công việc luôn thay đổi. Tương tự, Prof-gen [12] dựa vào các kỹ thuật tĩnh đòi hỏi cấu hình thủ công hoặc bỏ qua theo dõi động, hạn chế khả năng mở rộng và độ bền trong các triển khai thực tế.

KubeRosy [13] và Optimus [14] tích hợp eBPF để giám sát lời gọi hệ thống trực tiếp và cập nhật chính sách thời gian chạy. KubeRosy tận dụng Kubernetes CRD để tinh chỉnh chính sách theo thời gian thực nhưng vẫn bị ràng buộc với môi trường Kubernetes. Optimus sử dụng lọc dựa trên liên kết để thích nghi lập đi lập lại các hồ sơ lời gọi hệ thống, tuy nhiên tính tự động của nó bị hạn chế—yêu cầu sự phê duyệt của người vận hành đối với các lời gọi hệ thống không khớp và dừng cập nhật hồ sơ nếu phân tích liên kết thất bại. Prof-gen, mặc dù kết hợp theo dõi động lúc khởi động với phân tích đồ thị lời gọi tĩnh để tạo danh sách truy cập tối thiểu mà không cần mã nguồn, vẫn thiếu khả năng nắm bắt các thay đổi hành vi trong quá trình thực thi thời gian chạy.

B. Hệ Thống Phát Hiện Xâm Nhập Dựa Trên Bất Thường

Các kỹ thuật dựa trên học máy đã được nghiên cứu rộng rãi để phát hiện bất thường lời gọi hệ thống. IDS dựa trên BoSC sử dụng mô hình hóa Túi Lời Gọi Hệ Thống với các bộ phân loại như Rừng Ngẫu Nhiên, mang lại độ chính xác hợp lý trong môi trường quy mô nhỏ. Tuy nhiên, yêu cầu huấn luyện lại thường xuyên cản trở khả năng mở rộng và thích nghi.

Các mô hình nhẹ như Bộ Phát Hiện Thoát dựa trên TinyML nhắm đến các thiết bị IoT và biên nhưng không đủ khả năng xử lý các véc-tơ tấn công phức tạp trong các hệ thống đám mây hiệu suất cao do tính thích nghi và độ chính xác hạn chế.

DeSFAM giải quyết những hạn chế này trong Giai đoạn 2 thông qua bộ phát hiện bất thường lai kết hợp Bộ Mã Hóa Tự Động Biến Phân (VAE) với Rừng Cô Lập, được huấn luyện trên tập dữ liệu DongTing [10]. Phương pháp này cho phép phát hiện chính xác, độ trễ thấp đối với các cuộc tấn công zero-day mà không cần huấn luyện lại, đồng thời hỗ trợ suy luận cửa sổ trượt thời gian thực.

C. Các Phương Pháp Phát Hiện Dựa Trên Xuất Xứ

Các hệ thống hướng xuất xứ như PACED giám sát các mối quan hệ lời gọi hệ thống và quan hệ nhân quả giữa

Bảng I: So sánh các giải pháp bảo mật container tiên tiến.

Proposed Method	Year	Problem Addressed (Syscall/AD)	Limitations	Tools/Techniques Used
Optimus [27]	2024	Dynamic syscall policy logic	Relies on operator input and halts updates if associations fail	eBPF, Association filtering
Hybrid Syscall Profiling [28]	2024	Static-dynamic syscall profiling gaps	Runtime complexity increases in high-density deployments	Static + dynamic + eBPF
KubeRosy [29]	2024	Lack of real-time syscall enforcement	Restricted to Kubernetes environments	eBPF, LSM, Dynamic updates
TinyML Escape Detector [30]	2024	Container escape in resource-constrained IoT	Model aging and low scalability in cloud	TinyML, Semantic syscall modeling
Confine [31]	2023	Over-permissive syscall policies	False positives, static-only nature	Static analysis, Seccomp
Condo [32]	2023	Kernel permission leakage	Hardware-dependent (TEE required)	Kernel integrity checks, TEE
C2C [33]	2022	Syscall overexposure via configuration files	Manual input needed for precise filtering	Static analysis, YAML filtering
PACED [34]	2022	Cross-container privilege abuse	Overhead from continuous provenance monitoring	eBPF, Provenance tracking
Anomaly-based IDS using BoSC [35]	2021	Anomalous syscall detection	Requires retraining to adapt to new attack patterns	ML, BoSC, Sliding Window
Prof-gen [36]	2021	Static syscall surface minimization	Fails to detect dynamic behavior shifts	Static ELF, Tracing
Sprofiler [37]	2021	Syscall profiling for container isolation	Prone to false positives in noisy environments	Static + dynamic analysis

các container để phát hiện các cuộc tấn công xuyên không gian tên. Mặc dù hiệu quả, chúng phải chịu chi phí cao do xây dựng đồ thị liên tục và khả năng mở rộng hạn chế.

DeSFAM vượt qua điều này bằng cách nhúng ngữ cảnh lời gọi hệ thống (ví dụ: PID, không gian tên, dấu thời gian) trực tiếp vào các đặc trưng mô hình, cho phép nhận thức hành vi mà không phải chịu chi phí của việc theo dõi xuất xứ đầy đủ.

D. Các Cải Tiến Bảo Mật Cấp Kernel

Các biện pháp phòng thủ tầng kernel như Condo sử dụng Môi Trường Thực Thi Đáng Tin Cậy (TEE) để bảo vệ tính toàn vẹn quyền hạn. Mặc dù an toàn, sự phụ thuộc vào phần cứng chuyên dụng và sửa đổi kernel khiến chúng không phù hợp cho các triển khai đám mây quy mô lớn.

Ngược lại, DeSFAM đạt được tính di động và khả năng mở rộng bằng cách sử dụng các bản đồ eBPF và hook LSM để thực thi các chính sách lời gọi hệ thống một cách động, mà không cần sửa đổi kernel hay phụ thuộc vào TEE.

E. Các Hạn Chế Chung Của Các Phương Pháp Hiện Tại

Mặc dù đã có những tiến bộ đáng kể trong việc tối thiểu hóa lời gọi hệ thống và bảo mật container, các giải pháp hiện tại vẫn đối mặt với một số hạn chế chính:

- Các chính sách tĩnh không thể thích nghi với các hành vi thời gian chạy luôn thay đổi.
- Thiếu thực thi thời gian thực cho phép lạm dụng lời gọi hệ thống giữa các lần cập nhật.
- Các phương pháp học máy yêu cầu huấn luyện lại thường xuyên, hạn chế khả năng mở rộng.
- Các phụ thuộc phần cứng (ví dụ: TEE) hạn chế tính linh hoạt trong triển khai.

F. DeSFAM: Thúc Đẩy Trình Độ Hiện Tại

DeSFAM giải quyết các hạn chế chính của các hệ thống trước thông qua một kiến trúc đa giai đoạn thống nhất. Nó kết hợp lập hồ sơ lời gọi hệ thống lai, kết hợp phân tích phân tĩnh với theo dõi thời gian chạy động, để tạo ra các danh sách truy cập lời gọi hệ thống chính xác. Thực thi thời gian thực được thực hiện thông qua các probe eBPF và hook LSM, cho phép chặn độ trễ thấp và cập nhật chính sách mà không cần khởi động lại container. Phát hiện bất thường được xử lý bởi SyscallAD, một mô hình lai sử dụng Bộ Mã Hóa Tự Động Biến Phân và Rừng Cô Lập để phát hiện chính xác, không cần huấn luyện lại đối với các cuộc tấn công đã biết và zero-day. DeSFAM cũng tích hợp lọc nhận thức CVE và chấm điểm rủi ro dựa trên MITRE ATT&CK để hạn chế động các lời gọi hệ thống dễ bị tổn thương, đồng thời áp dụng logic phản hồi phân tầng để thực thi các hành động chi tiết dựa trên mức độ nghiêm trọng của mối đe dọa. Khung hoạt động mà không cần sửa đổi kernel hay phụ thuộc vào các nền tảng điều phối, cung cấp bảo mật lời gọi hệ thống có khả năng mở rộng và thích nghi cho các môi trường container hiện đại.

IV. DESFAM: LỌC SYSCALL ĐỘNG DỰA TRÊN EBPF VÀ GIẢM THIỂU BẤT THƯỜNG

Phần này trình bày thiết kế và triển khai của framework DeSFAM được đề xuất, bao gồm kiến trúc, các giai đoạn hoạt động và chiến lược thực thi.

A. Tổng Quan

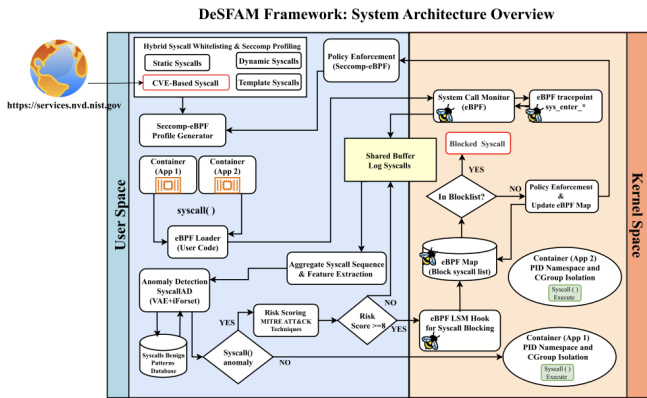
DeSFAM (Dynamic eBPF-Driven Syscall Filtering and Anomaly Mitigation) là một framework nhẹ, hoạt động thời gian thực, được thiết kế để thu hẹp bề mặt tấn công

kernel và phát hiện hành vi syscall bất thường trong các môi trường container. Framework thực thi nguyên tắc đặc quyền tối thiểu trong sử dụng syscall nhằm giảm thiểu các mối đe dọa như leo thang đặc quyền, thoát container và chèn syscall. Hình 1 minh họa quy trình kiến trúc của framework.

DeSFAM hoạt động theo ba giai đoạn tích hợp:

- **Lập Danh Sách Truy Cập Syscall Kết Hợp và Tạo Hồ Sơ Seccomp:** Kết hợp trích xuất đồ thị lời gọi tinh và theo dõi eBPF động để xây dựng danh sách truy cập syscall được tinh chỉnh, loại bỏ các syscall không cần thiết và liên quan đến CVE.
- **Giám Sát Thời Gian Thực và Phát Hiện Bất Thường (SyscallAD):** Theo dõi các chuỗi syscall thời gian thực và phân tích chúng bằng tập hợp VAE + Isolation Forest để phát hiện các bất thường ở mức chuỗi, bao gồm cả việc sử dụng độc hại các syscall được phép.
- **Thực Thi Thích Ứng và Quản Lý Chính Sách:** Thực thi việc chặn, giới hạn tốc độ hoặc ghi nhật ký các syscall có rủi ro cao bằng eBPF maps và LSM hooks, dựa trên điểm rủi ro từ SyscallAD, mà không làm gián đoạn quá trình thực thi container.

Bằng cách tích hợp lập hồ sơ nhận biết chuỗi, phát hiện bất thường và thực thi thời gian thực, DeSFAM cung cấp khả năng bảo vệ có thể mở rộng và thích ứng, phù hợp với các triển khai container cloud-native hiện đại.



Hình 1: Kiến trúc DeSFAM thể hiện sự tương tác giữa lập hồ sơ syscall ở không gian người dùng, phát hiện bất thường và thực thi ở không gian kernel sử dụng eBPF và seccomp cho điều khiển syscall thích ứng, thời gian thực.

B. Các Cân Nhắc Thiết Kế

Thiết kế của DeSFAM được định hướng bởi nhu cầu thực thi nguyên tắc đặc quyền tối thiểu cho syscall trong khi giải quyết các hạn chế của lọc chỉ tĩnh, theo dõi xuất xứ chi phí cao và các bộ phát hiện bất thường phụ thuộc vào việc huấn luyện lại. Các hồ sơ seccomp truyền thống hoặc quá dễ dãi hoặc quá cứng nhắc, không thể thích nghi với hành vi thời gian thực đang thay đổi. DeSFAM khắc phục điều này bằng cách kết hợp phân tích tĩnh và động,

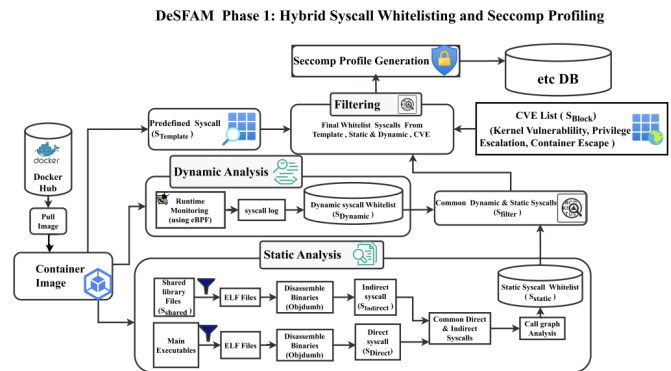
cho phép lập hồ sơ syscall chính xác và nhận thức ngữ cảnh.

Để đảm bảo thực thi thời gian thực mà không ảnh hưởng đến tính khả dụng của container, DeSFAM tận dụng các eBPF probes và LSM hooks, hoạt động hoàn toàn trong không gian kernel và hỗ trợ cập nhật chính sách động với độ trễ tối thiểu. Lựa chọn thiết kế này tránh chi phí hiệu năng của việc xây dựng đồ thị xuất xứ liên tục và loại bỏ nhu cầu sửa đổi kernel hoặc khởi động lại container.

Đối với phát hiện bất thường, DeSFAM áp dụng mô hình kết hợp (VAE + Isolation Forest) nắm bắt các mẫu hành vi trong chuỗi syscall trong khi tránh huấn luyện lại thường xuyên, hỗ trợ phát hiện có thể mở rộng và mạnh mẽ các cuộc tấn công zero-day. Không giống các giải pháp bị ràng buộc phần cứng như TEEs, DeSFAM vẫn hoàn toàn dựa trên phần mềm và có thể di chuyển, đảm bảo tính tương thích trên nhiều triển khai cloud-native khác nhau.

C. Giai Đoạn 1: Lập Danh Sách Truy Cập Syscall Kết Hợp và Tạo Hồ Sơ Seccomp

Bảo mật container phụ thuộc vào kiểm soát syscall chính xác, vì các container dùng chung kernel của máy chủ và đặt ra rủi ro truy cập kernel trái phép. Giai đoạn 1 của DeSFAM giảm thiểu điều này bằng cách xây dựng danh sách truy cập syscall dành riêng cho container thông qua phương pháp kết hợp kết hợp phân tích tĩnh, lập hồ sơ động và thông tin tình báo mối đe dọa dựa trên CVE để tạo ra hồ sơ seccomp tối thiểu và chính xác.



Hình 2: DeSFAM Giai Đoạn 1: Lập danh sách truy cập syscall kết hợp và tạo hồ sơ seccomp sử dụng phân tích tĩnh, giám sát eBPF và lọc thông tin CVE để tạo ra các chính sách syscall an toàn dành riêng cho container.

1) *Phân Tích Tĩnh*: Phân tích tĩnh bắt đầu bằng cách kiểm tra các image container, bao gồm Dockerfile, binary và các thư viện được liên kết, mà không thực thi chúng. Sử dụng các công cụ như `objdump` và `readelf`, các binary ELF được dịch ngược để trích xuất các lệnh syscall cấp thấp và các hàm wrapper của libc.

Để tăng cường độ bao phủ, một Đồ Thị Lời Gọi Tĩnh (SCG) được xây dựng bằng NetworkX để theo dõi ánh xạ từ hàm sang syscall trên toàn cơ sở mã. Tập syscall tĩnh

kết quả, S_{static} , nắm bắt tất cả các syscall có thể xác định được từ luồng điều khiển của mã.

- **Đầu vào:** Image container (Dockerfile, ELF binary)
- **Công cụ:** `objdump`, `readelf`, `networkx`
- **Quá trình:** Dịch ngược ELF binary, trích xuất đường dẫn syscall qua duyệt SCG
- **Đầu ra:** Danh sách syscall tĩnh (S_{static})

Tuy nhiên, phân tích tĩnh không thể phát hiện các syscall được gọi bởi các thư viện được giải quyết lúc chạy hoặc mã được biên dịch JIT. Do đó, trong khi S_{static} cung cấp một tập syscall nền tảng, nó phải được tinh chỉnh thông qua phân tích động trong các giai đoạn tiếp theo.

2) *Lập Hồ Sơ Động: Nắm Bắt Hành Vi Thời Gian Thực:*

Để giải quyết các hạn chế của phân tích tĩnh, DeSFAM kết hợp lập hồ sơ động bằng cách giám sát hoạt động syscall trực tiếp trong quá trình thực thi container dưới các khối lượng công việc đại diện chuẩn hóa. Giai đoạn này nắm bắt các syscall được kích hoạt bởi các điều kiện thời gian thực bao gồm các thư viện được tải động, logic đặc thù môi trường và các tương tác phụ thuộc, đồng thời đảm bảo độ bao phủ toàn diện các hành vi vận hành điển hình.

Các container thực thi dưới các khối lượng công việc dành riêng cho vai trò được thiết kế bằng các benchmark tiêu chuẩn ngành phản ánh các hồ sơ vận hành thực tế:

- **Máy chủ web (Nginx):** `wrk` với 500–10,000 kết nối đồng thời; ApacheBench mô phỏng các yêu cầu GET/POST tại 1K–10K req/sec.
- **Cơ sở dữ liệu (MySQL/Redis/MongoDB):** Khối lượng công việc SysBench OLTP (100–500 luồng, 1M+ giao dịch); `redis-benchmark` tại 1M ops/sec.
- **Thời gian chạy ứng dụng (Node.js/Python):** Kiểm tra tải Locust mô phỏng 50–500 luồng người dùng đồng thời; xử lý tập dữ liệu thực tế (phân tích JSON/CSV quy mô GB).

Các khối lượng công việc này bao gồm các mẫu lưu lượng sản xuất ở phân vị thứ 90, chuỗi khởi động/tắt container, tương tác phụ thuộc và các kịch bản lỗi.

Trong quá trình thực thi, các eBPF probes được gắn vào các điểm vào syscall (`sys_enter_*`) để theo dõi các sự kiện syscall. Siêu dữ liệu được thu thập bao gồm ID syscall, ID tiến trình, dấu thời gian và định danh namespace.

- **Đầu vào:** Các instance container dưới các khối lượng công việc chuẩn hóa
- **Công cụ:** eBPF qua `bcc/bpftrace` với theo dõi ở không gian kernel
- **Quá trình:** Triển khai container → gắn eBPF probes → ghi siêu dữ liệu vào ring buffers → tổng hợp syscall qua 3 chu kỳ khối lượng công việc
- **Đầu ra:** Danh sách syscall động ($S_{dynamic}$)

3) *Thông Tin Tình Báo Mỗi Đe Dọa: Lọc Syscall Dựa trên CVE:* Để tăng cường nhận thức bảo mật, DeSFAM tích hợp thông tin tình báo mỗi đe dọa bằng cách kết hợp dữ liệu CVE từ Cơ Sở Dữ Liệu Lỗ Hổng Quốc Gia (NVD). Sử dụng xử lý ngôn ngữ tự nhiên (NLP), chúng tôi trích xuất tên syscall (ví dụ: `ptrace`, `clone`, `execve`) từ các mô tả CVE để xây dựng danh sách syscall có rủi ro cao, ký hiệu là $S_{blocked}$.

Để xây dựng $S_{blocked}$, chúng tôi sử dụng một công cụ phân tích dựa trên NLP phân tích các báo cáo CVE được lấy từ NVD qua REST API của nó. Kỹ thuật trích xuất từ khóa dựa trên quy tắc được áp dụng để xác định tên syscall được đề cập trong các mô tả CVE. Chúng tôi sử dụng từ điển được chọn lọc của tên syscall Linux có nguồn gốc từ các header nguồn kernel (ví dụ: `unistd64.h`) để đảm bảo ánh xạ chính xác. Hơn nữa, để tránh chặn quá mức, các siêu dữ liệu ngữ cảnh như khả năng áp dụng phiên bản kernel, các hệ thống con bị ảnh hưởng, điều kiện khai thác và ràng buộc tham số được trích xuất từ văn bản CVE bằng cách khớp mẫu và phân tích biểu thức chính quy.

- **Đầu vào:** Các mục CVE có đề cập syscall
- **Ảnh xạ:** Trích xuất syscall dựa trên quy tắc + lọc ngữ cảnh
- **Đầu ra:** Danh sách syscall bị chặn ($S_{blocked}$)

Tập này được loại trừ khỏi hợp của các tập syscall tĩnh và động, đảm bảo các syscall dễ bị tổn thương bị bỏ qua ngay cả khi được quan sát trong quá trình lập hồ sơ.

4) *Tổng Hợp Danh Sách Truy Cập Cuối Cùng và Tạo Chính Sách:* Danh sách truy cập cuối cùng S_{final} được tính như sau:

$$S_{final} = (S_{static} \cup S_{dynamic}) \cap S_{template} \setminus S_{blocked} \setminus S_{template} \quad (1)$$

Ở đây, $S_{template}$ định nghĩa hành vi syscall mong đợi dành riêng cho vai trò của container (ví dụ: máy chủ web, cơ sở dữ liệu, khối lượng công việc GPU), đóng vai trò là ranh giới chức năng. Các syscall template được suy ra bằng cách kết hợp nhiều nguồn: hồ sơ seccomp mặc định chính thức của Docker, tài liệu syscall kernel Linux, các quy tắc bảo mật do cộng đồng duy trì từ dự án Falco, và các dấu vết syscall thực nghiệm được thu thập trong quá trình thực thi container lành mạnh ban đầu.

Phương pháp đa nguồn này đảm bảo rằng các syscall hiếm khi được gọi nhưng cần thiết trong quá trình khởi tạo container hoặc các đường dẫn thực thi không phổ biến được đưa vào $S_{template}$ trong khi vẫn duy trì mức tiếp xúc syscall tối thiểu.

Tập syscall dựa trên CVE $S_{blocked}$ được tạo ra bằng cách trích xuất các điều kiện khai thác, bao gồm phiên bản kernel, các lỗ hổng dành riêng cho thiết bị, hoặc các mẫu tham số syscall cụ thể. Điều này ngăn chặn việc chặn quá mức các syscall chỉ dễ bị tổn thương trong các cấu hình cụ thể không áp dụng cho môi trường thời gian chạy của container.

Danh sách truy cập kết quả S_{final} được biên dịch thành chính sách seccomp (`seccomp_profile.json`) được thực thi bởi kernel Linux lúc chạy. Hình 2 và Thuật toán 1 tóm tắt quá trình này.

5) *Tải Chính Sách Dành Riêng cho Container:*

DeSFAM tải động các chính sách syscall dành riêng cho container bằng `policy_loader`. Mỗi chính sách phản ánh hồ sơ hành vi của container từ phân tích trước đó và được ghi vào eBPF map được ghim tại `/sys/fs/bpf/blocked_syscalls`. Định danh namespace đóng vai trò là khóa để cách ly container.

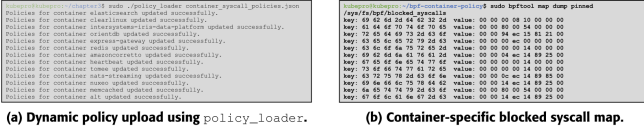
Algorithm 1 Lập Hồ Sơ Syscall Kết Hợp Nâng Cao

Đầu vào: Image container, instance đang chạy, cơ sở dữ liệu CVE

Đầu ra: Danh sách truy cập syscall an toàn và có chức năng

- 1: Trích xuất syscall qua phân tích tĩnh $\rightarrow S_{static}$
- 2: Lập hồ sơ thời gian thực dưới khối lượng công việc (lập hồ sơ động) $\rightarrow S_{dynamic}$
- 3: Phân tích CVE với phân tích ngữ cảnh $\rightarrow S_{blocked}$
- 4: Tạo $S_{final} = (S_{static} \cup S_{dynamic}) \cap S_{template} \setminus S_{blocked} \setminus S_{template}$
- 5: **return** S_{final}

Thiết kế này thực thi truy cập syscall đặc quyền tối thiểu cho mỗi container lúc chạy, mà không cần khởi động lại instance. Hình 3 cho thấy quy trình tải và việc điền vào Syscall map.



Hình 3: DeSFAM Giai Đoạn 3: (a) Tải động các chính sách syscall dành riêng cho container sử dụng `policy_loader`; (b) Xác minh các syscall bị chặn trong eBPF map được ghim sử dụng `bpftool`.

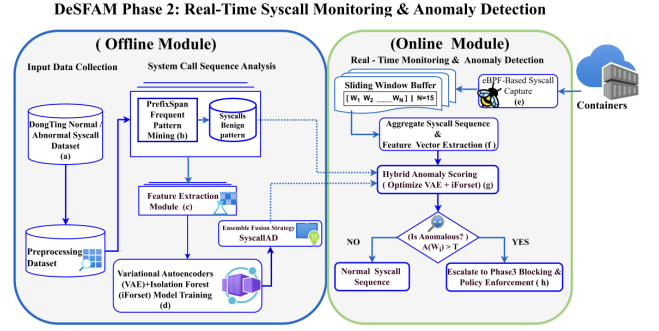
D. Giai Đoạn 2: Giám Sát Thời Gian Thực và Phát Hiện Bất Thường Syscall (SyscallAD)

Giai đoạn 2 của DeSFAM giới thiệu SyscallAD, một module phát hiện bất thường thời gian thực kết hợp học sâu và các phương pháp tập hợp để xác định cả các mối đe dọa đã biết và zero-day trong các môi trường container. Nó giám sát các chuỗi syscall bằng eBPF và phát hiện các độ lệch hành vi liên quan đến leo thang đặc quyền, thoát container hoặc chèn syscall.

SyscallAD sử dụng một tập hợp kết hợp gồm Bộ Mã Hóa Biến Phân (VAE), học các biểu diễn ẩn của các mẫu syscall lành mạnh, và Isolation Forest (iForest), xác định các điểm ngoại lệ thống kê. Sự kết hợp này cho phép phát hiện mạnh mẽ, độ trễ thấp với ít dương tính giả và chi phí tối thiểu.

1) *Bộ Dữ Liệu DongTing: Đường Cơ Sở Hành Vi cho Phát Hiện:* SyscallAD được huấn luyện trên bộ dữ liệu DongTing [10], gồm 18,966 chuỗi syscall và hơn 2,6 triệu syscall riêng lẻ được thu thập từ các ứng dụng container. Bộ dữ liệu được chia thành:

- **6,850 chuỗi lành mạnh:** Được thu thập từ các khối lượng công việc bình thường như máy chủ web và cơ sở dữ liệu thực hiện I/O tệp (`open`, `read`) và các thao tác bộ nhớ (`mmap`, `futex`).
- **12,116 chuỗi độc hại:** Được tạo ra thông qua các cuộc tấn công mô phỏng, bao gồm leo thang đặc



Hình 4: DeSFAM Giai Đoạn 2: Giám sát syscall thời gian thực và phát hiện bất thường sử dụng mô hình kết hợp VAE + iForest. Module ngoại tuyến được huấn luyện trên các chuỗi DongTing, trong khi module trực tuyến xử lý các syscall trực tiếp qua eBPF để đánh giá bất thường và leo thang chính sách.

quyền (`setuid`), giả mạo hệ thống tệp (`unlink`) và xâm nhập mạng (`socket`).

Mỗi chuỗi bao gồm ID syscall, dấu thời gian và siêu dữ liệu ngữ cảnh (ví dụ: ID tiến trình, namespace). Với sự mất cân bằng lớp vốn có, các kỹ thuật lấy mẫu được áp dụng để ngăn ngừa thiên lệch mô hình.

2) *Khai Thác Mẫu PrefixSpan: Trích Xuất Các Template Syscall Đáng Tin Cậy:* Để giảm thiểu dương tính giả và cải thiện hiệu quả phát hiện, DeSFAM tích hợp PrefixSpan [15]—một thuật toán khai thác mẫu tuần tự—trên tập con lành mạnh của bộ dữ liệu DongTing. Điều này xác định các chuỗi con syscall thường xuyên đại diện cho hành vi container hợp lệ.

Cơ Sở Dữ Liệu Mẫu Lành Mạnh kết quả được sử dụng để:

- Khớp các cửa sổ syscall đến với các mẫu đã biết là tốt để phân loại lành mạnh sớm.
- Bỏ qua chấm điểm bất thường sâu cho các chuỗi khớp, giảm chi phí tính toán.
- Cung cấp khả năng truy xuất ngữ nghĩa cho các quyết định bảo mật.

3) *Kỹ Thuật Đặc Trưng: Cấu Trúc Hóa Chuỗi Syscall cho Phát Hiện:* SyscallAD áp dụng kỹ thuật đặc trưng đa chiều để chuyển đổi các luồng syscall thô thành các đầu vào mô hình có thể hành động:

- **Cửa Sổ Trượt:** Các syscall được phân đoạn thành các cửa sổ chồng lấp (độ dài 15, bước 3), nắm bắt cả các bất thường ngắn hạn và các mẫu hành vi mở rộng.
- **Tần Suất Danh Mục:** Các syscall được ánh xạ tới các danh mục chức năng (ví dụ: I/O tệp, bộ nhớ, mạng) và tần suất của chúng mỗi cửa sổ tạo thành chữ ký hành vi.
- **Đặc Trưng Thời Gian:** Các khoảng thời gian giữa syscall (delta thời gian) được tóm tắt (trung bình, độ lệch chuẩn, giá trị lớn nhất) để phát hiện các hành vi thời gian bất thường.

- **Khớp Mẫu Danh Sách Truy Cập:** Mỗi cửa sổ được kiểm tra chéo với các mẫu lành mạnh từ PrefixSpan; các kết khớp làm tăng độ tin cậy trong phân loại lành mạnh, giảm dương tính giả.
- **Tăng Cường Dữ Liệu:** Các chuỗi độc hại được sửa đổi tổng hợp (ví dụ: chèn hoặc hoán vị các syscall rủi ro cao như `clone`, `setuid`) để mô phỏng các kịch bản zero-day, cải thiện độ mạnh mẽ của mô hình.

4) *SyscallAD: Engine Học Sâu Kết Hợp và Phát Hiện Bất Thường:* SyscallAD là một module phát hiện bất thường kết hợp được thiết kế để xác định các độ lệch hành vi trong chuỗi syscall với độ chính xác và khả năng thích ứng cao.

a) *Bộ Mã Hóa Biến Phân Được Tối Ưu Hóa (VAE):* VAE sử dụng kiến trúc encoder-decoder ($32 \rightarrow 8 \rightarrow 32$ neurons) với các kích hoạt SELU, dropout ($p = 0.2$), và chính quy hóa L2. Nó học cách tái tạo các cửa sổ syscall lành mạnh bằng cách tối thiểu hóa Lỗi Tái Tạo Bình Phương Trung Bình (MSRE). Các bất thường được gắn cờ khi MSRE vượt quá phân vị thứ 99.5 của các lỗi huấn luyện lành mạnh.

b) *Isolation Forest (iForest):* iForest cô lập các bất thường thông qua phân vùng đệ quy. Các chuỗi syscall hiếm gặp tạo ra các cây cô lập nông hơn và điểm bất thường cao hơn. Tham số nhiệm là 2% kiểm soát độ nhạy, cho phép phát hiện các hành vi hiếm về mặt thống kê bị VAE bỏ qua.

c) *Chiến Lược Hợp Nhất Tập Hợp:* SyscallAD kết hợp các điểm mạnh của VAE và Isolation Forest thông qua cơ chế hợp nhất điểm có trọng số. Mỗi cửa sổ syscall W_i được đánh giá bởi cả hai mô hình:

- $A_{VAE}(W_i)$: Điểm bất thường dựa trên Lỗi Tái Tạo Bình Phương Trung Bình.
- $A_{iForest}(W_i)$: Điểm xuất phát từ độ sâu cô lập trong Isolation Forest.

Điểm cuối cùng được tính như sau:

$$A(W_i) = \alpha \cdot A_{VAE}(W_i) + (1 - \alpha) \cdot A_{iForest}(W_i) \quad (2)$$

trong đó $\alpha = 0.7$ ưu tiên đóng góp của VAE. Giá trị này được xác nhận qua phân tích độ nhạy trên $\alpha \in \{0.0, 0.25, 0.5, 0.7, 0.85, 1.0\}$ trên bộ dữ liệu DongTing, như thể hiện trong Hình 5.

Hình 6 trình bày nhật ký bất thường từ các container `test-mysql` và `test-redis`. Các syscall lành mạnh như `futex`, `epoll_wait` và `wait4` được phân loại chính xác, trong khi các bất thường như `clone` và `ioctl`, chỉ ra các nỗ lực thoát hoặc leo thang đặc quyền, được gắn cờ hiệu quả.

d) *Ngưỡng Động: Điều chỉnh Ranh Giới Bất Thường Thích Ứng:* Để phù hợp với hành vi syscall đang thay đổi, SyscallAD sử dụng ngưỡng động dựa trên Trung Bình Động Hàm Mũ (EMA), thay vì một điểm cắt cố định. Ngưỡng ban đầu T_0 được đặt ở phân vị thứ 99.5 của các lỗi tái tạo VAE trên dữ liệu xác nhận lành mạnh. Trong thời gian chạy, ngưỡng được cập nhật sau mỗi cửa sổ syscall W_i như sau:

$$T_{t+1} = \max(T_{\min}, \beta T_t + (1 - \beta)A(W_i)) \quad (3)$$

trong đó $\beta = 0.9$ kiểm soát độ nhạy cập nhật và $T_{\min} = 0.5$ thực thi giới hạn dưới.

Phát hiện cuối cùng được định nghĩa là:

$$\text{Flag}(W_i) = \begin{cases} 1, & \text{nếu } A(W_i) > T \\ 0, & \text{ngược lại} \end{cases} \quad (4)$$

5) *Giám Sát Thời Gian Thực với eBPF: Thu Thập Syscall Nhận Biết Container Chi Phí Thấp:* DeSFAM sử dụng các eBPF tracepoints để cho phép giám sát syscall chi tiết, chi phí thấp trực tiếp ở mức kernel Linux. Chương trình BPF móc vào (`raw_syscalls:sys_enter`) và (`raw_syscalls:sys_exit`), cho phép chặn các lời gọi syscall tại các điểm vào và thoát của chúng.

Để cô lập hoạt động syscall từ các khối lượng công việc container, DeSFAM ánh xạ động các PID container bằng cách truy vấn Docker runtime và chèn chúng vào bảng băm BPF `pid_map`. Chỉ các syscall có nguồn gốc từ các PID này mới được ghi lại, đảm bảo theo dõi dành riêng cho container mà không có nhiễu hệ thống không cần thiết.

Các sự kiện syscall được nắm bắt được làm giàu với siêu dữ liệu ngữ cảnh bao gồm: ID syscall, dấu thời gian (độ phân giải nano giây), PID tiến trình và cha, tên lệnh, tên container và giá trị trả về syscall. Chúng được gửi từ kernel đến không gian người dùng qua `ring_buffer` và được ghi vào các tệp theo container ở định dạng JSON. Đồng thời, các ID syscall được thêm vào một deque cuộn (cửa sổ trượt kích thước 15) trong bộ nhớ cho mỗi container. Hình 7 cho thấy một nhật ký syscall thời gian thực đại diện cho một container MySQL.

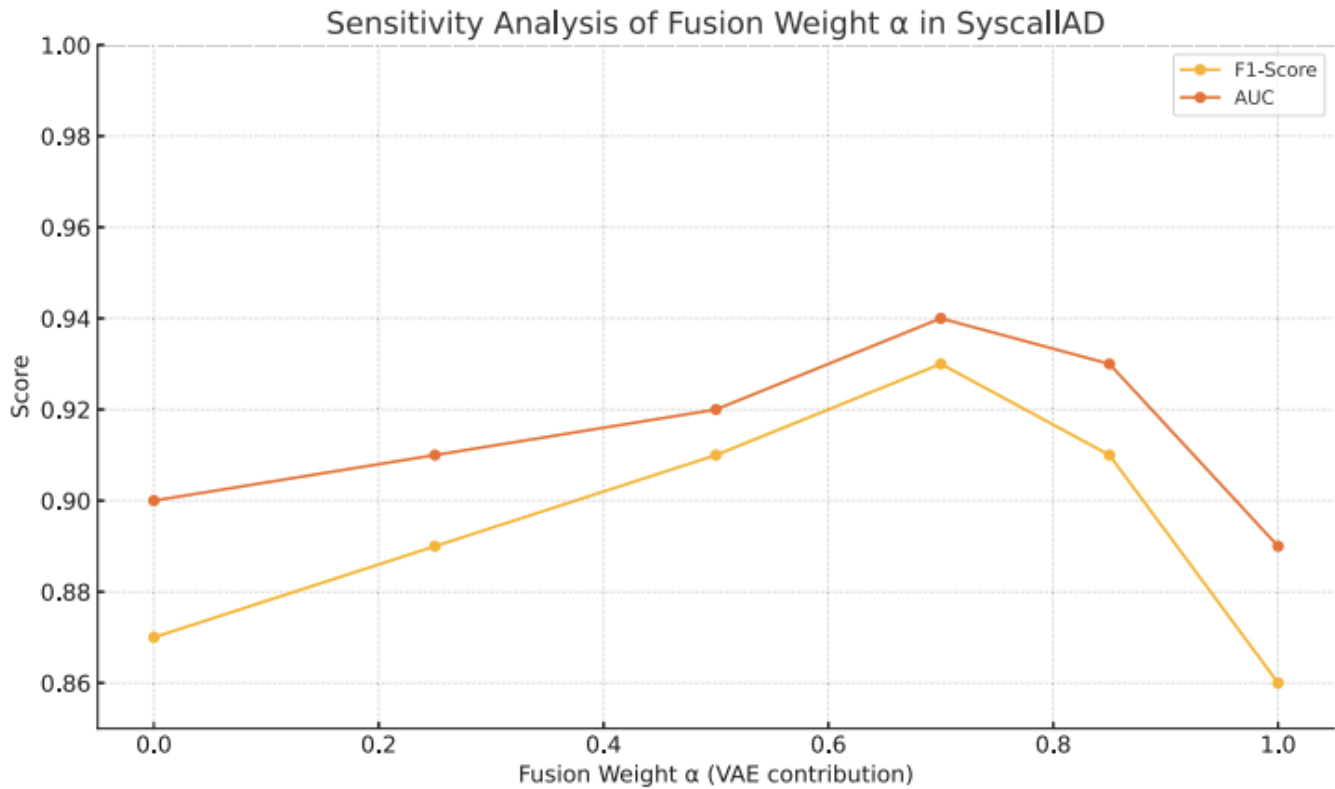
6) *Tích Hợp DevOps: Tinh Chỉnh Mô Hình Liên Tục:* Để giảm thiểu dương tính giả và thích nghi với các khối lượng công việc đang thay đổi, DeSFAM tích hợp vòng phản hồi theo hướng DevOps trong Giai Đoạn 2. Các chuỗi syscall bất thường, cùng với siêu dữ liệu (ví dụ: tên container, PID, dấu thời gian), được ghi nhật ký và trực quan hóa để nhà phân tích xem xét. Các nhóm bảo mật có thể chú thích các phát hiện là dương tính thật hoặc dương tính giả, với các dương tính giả đã xác nhận được thêm vào tập huấn luyện lành mạnh.

SyscallAD định kỳ huấn luyện lại tập hợp VAE + iForest của mình trên bộ dữ liệu tăng cường này, cho phép nó học các biến thể lành mạnh mới được giới thiệu bởi các khối lượng công việc đặc thù môi trường hoặc các trường hợp ngoại lệ.

Quy trình phát hiện đầy đủ được hình thức hóa trong Thuật toán 2.

E. Giai Đoạn 3: Thực Thi Syscall Thích Ứng và Quản Lý Chính Sách

Thực Thi Syscall Thích Ứng là giai đoạn thứ ba của framework DeSFAM. Nó tập trung vào việc thực thi động các chính sách syscall trong các môi trường container theo thời gian thực. Giai đoạn này tích hợp chấm điểm rủi ro, phân tích ngữ cảnh và thông tin tình báo mối đe dọa bên ngoài để cung cấp các phản hồi bảo mật thích ứng và chính xác.



Hình 5: Phân tích độ nhạy của trọng số hợp nhất α trên F1-Score và AUC. Hiệu suất đỉnh được quan sát tại $\alpha = 0.7$.

```

2025-03-02T13:22:47.799890 | Normal behavior detected in container: test-mysql | Syscall: futex
2025-03-02T13:22:48.206957 | Normal behavior detected in container: test-mysql | Syscall: futex
2025-03-02T13:22:48.297612 | Normal behavior detected in container: test-redis | Syscall: epoll_wait
2025-03-02T13:22:48.350686 | Anomaly detected in container: test-redis | Syscall: clone
2025-03-02T13:22:49.578179 | Normal behavior detected in container: test-redis | Syscall: wait4
2025-03-02T13:22:49.586038 | Normal behavior detected in container: test-redis | Syscall: wait4
2025-03-02T13:22:49.593676 | Anomaly detected in container: test-redis | Syscall: clone
2025-03-02T13:22:50.844151 | Normal behavior detected in container: test-mysql | Syscall: futex
2025-03-02T13:22:50.851959 | Normal behavior detected in container: test-redis | Syscall: wait4
2025-03-02T13:22:50.860505 | Normal behavior detected in container: test-mysql | Syscall: futex
2025-03-02T13:22:51.912785 | Normal behavior detected in container: test-mysql | Syscall: futex
2025-03-02T13:22:51.970742 | Normal behavior detected in container: test-redis | Syscall: wait4
2025-03-02T13:22:51.978846 | Normal behavior detected in container: test-mysql | Syscall: io_getevents
2025-03-02T13:22:52.033732 | Normal behavior detected in container: test-mysql | Syscall: io_getevents
2025-03-02T13:22:52.233711 | Normal behavior detected in container: test-redis | Syscall: wait4
2025-03-02T13:22:52.242228 | Normal behavior detected in container: test-redis | Syscall: wait4
2025-03-02T13:22:53.251668 | Anomaly detected in container: test-redis | Syscall: ioctl
2025-03-02T13:22:53.324820 | Normal behavior detected in container: test-redis | Syscall: epoll_wait
2025-03-02T13:22:53.392566 | Normal behavior detected in container: test-redis | Syscall: wait4

```

Hình 6: Đầu ra SyscallAD từ DeSFAM Giai Đoạn 2: hành vi bình thường trong test-mysql tương phản với các phát hiện syscall bất thường trong test-redis (ví dụ: clone, ioctl).

```

{"timestamp": "2025-01-28T07:34:10.602445Z", "syscall_id": 202, "pid": 7593,
"ppid": 8672, "container": "kubepro-mysql-1", "user": "ib_log_files_g",
"return_value": 0}

{"timestamp": "2025-01-28T07:34:10.602446Z", "syscall_id": 0, "pid": 7593, "ppid":
0, "container": "kubepro-mysql-1", "user": "ib_log_files_g", "return_value": 0}

{"timestamp": "2025-01-28T07:34:10.602456Z", "syscall_id": 202, "pid": 7593,
"ppid": 8672, "container": "kubepro-mysql-1", "user": "ib_log_files_g",
"return_value": 0}

{"timestamp": "2025-01-28T07:34:10.612529Z", "syscall_id": 0, "pid": 7593, "ppid":
0, "container": "kubepro-mysql-1", "user": "ib_log_files_g", "return_value":
18446744073709551506}

{"timestamp": "2025-01-28T07:34:10.612544Z", "syscall_id": 202, "pid": 7593,
"ppid": 8672, "container": "kubepro-mysql-1", "user": "ib_log_files_g",
"return_value": 0}

```

Hình 7: Nhật ký syscall eBPF thời gian thực cho kubepro-mysql-1, hiển thị các sự kiện syscall có dấu thời gian với PID, tên container, người dùng và giá trị trả về.

Algorithm 2 Phát Hiện Bất Thường Syscall Thời Gian Thực (SyscallAD)

Đầu vào: eBPF hook, các mô hình VAE và iForest đã huấn luyện, các lỗi tái tạo xác nhận E_{val}

Đầu ra: Cờ bất thường cho mỗi cửa sổ syscall

```

1: Khởi tạo các cửa sổ trượt  $W_i \leftarrow \emptyset$  cho  $i \in \{5, 10, 15\}$ 
2: Tải các mô hình VAE và iForest đã huấn luyện
3: Tính  $T_0 \leftarrow \text{percentile}(E_{val}, 99.5)$ 
4: Đặt ngưỡng  $T \leftarrow T_0$ 
5: while luồng syscall đang hoạt động do
6:   Thu thập syscall  $s_t$  tại thời điểm  $t$ 
7:   Thêm  $s_t$  vào mỗi  $W_i$ 
8:   for mỗi cửa sổ đầy  $W_i$  do
9:      $f_i \leftarrow \text{extract\_features}(W_i)$ 
10:     $A_{VAE} \leftarrow \text{VAE\_score}(f_i)$ 
11:     $A_{iForest} \leftarrow \text{iForest\_score}(f_i)$ 
12:     $A(W_i) \leftarrow 0.7 \cdot A_{VAE} + 0.3 \cdot A_{iForest}$ 
13:    if  $A(W_i) > T$  then
14:      Gắn cờ  $W_i$  là bất thường; leo thang lên Giai
      Đoạn 3
15:    else
16:       $T \leftarrow \max(T_{min}, \beta \cdot T + (1 - \beta) \cdot A(W_i))$ 
17:    end if
18:    Trượt  $W_i$  về phía trước
19:  end for
20: end while

```

1) *Cơ Chế Chấm Điểm Rủi Ro*: DeSFAM định lượng mức độ đe dọa của mỗi chuỗi syscall bất thường W_i bằng điểm rủi ro tổng hợp được suy ra từ ba thành phần.

a) *Ánh Xạ MITRE ATT&CK*: Mỗi syscall trong W_i được ánh xạ tới các kỹ thuật đối thủ liên quan (ví dụ:

T1055 cho `ptrace`) dựa trên bảng tra cứu được xác định trước (Bảng II). Ánh xạ ngữ cảnh này cho phép suy luận vượt ra ngoài các lời gọi riêng lẻ để nhận ra các mẫu tấn công cấp cao.

b) *Mức Nghiêm Trọng Tích Lũy*: Điểm mức nghiêm trọng tích lũy tổng hợp các điểm mức nghiêm trọng riêng lẻ của các syscall trong chuỗi:

$$\text{Cumulative Severity} = \sum_{t \in W_i} \text{Severity}(t) \quad (5)$$

c) *Khớp Mẫu Khai Thác*: Mỗi chuỗi W_i được đánh giá đối chiếu với các chữ ký khai thác CVE đã biết. Nếu tìm thấy kết khớp, một hình phạt rủi ro bổ sung được áp dụng:

$$I(\text{Exploit Match}) = \begin{cases} 1, & \text{nếu khớp} \\ 0, & \text{ngược lại} \end{cases} \quad (6)$$

d) *Điểm Rủi Ro Cuối Cùng*: Điểm rủi ro cuối cùng tích hợp cả mức nghiêm trọng và thông tin tình báo khai thác:

$$\text{Risk Score} = \sum_{t \in W_i} \text{Severity}(t) + 5 \cdot I(\text{Exploit Match}) \quad (7)$$

e) *Logic Quyết Định Chính Sách*: Dựa trên điểm rủi ro được tính toán, DeSFAM áp dụng chiến lược thực thi ba cấp:

$$\text{Action} = \begin{cases} \text{Block}, & \text{nếu Risk Score} \geq 8 \\ \text{Rate-limit}, & \text{nếu } 5 \leq \text{Risk Score} < 8 \\ \text{Log}, & \text{nếu Risk Score} < 5 \end{cases} \quad (8)$$

Các ngưỡng này được điều chỉnh theo kinh nghiệm và vẫn có thể cấu hình cho các tư thế rủi ro đặc thù triển khai. Bảng III trình bày các chuỗi mẫu và các hành động tương ứng.

Bảng II: Ví dụ Ảnh Xạ Syscall-sang-Kỹ Thuật MITRE ATT&CK.

Syscall	Technique ID	Technique Name	Tactic	Severity (1-10)
ptrace	T1055	Process Injection	Defense Evasion	9
execve	T1059	Command and Scripting Interpreter	Execution	8
clone	T1055.012	Thread Execution Hijacking	Defense Evasion	7
setuid	T1548	Abuse Elevation Control Mechanism	Privilege Escalation	8
mprotect	T1027	Obfuscated Files or Information	Defense Evasion	7
socket	T1071	Application Layer Protocol	Command and Control	6
kill	T1561	Interrupt Inhibitor	Impact	5
unshare	T1611	Escape to Host	Privilege Escalation	9
mount	T1006	Direct Volume Access	Persistence	8
chroot	T1610	Deploy Container	Defense Evasion	8
usersn	T1068	Exploitation for Privilege Escalation	Privilege Escalation	9
setns	T1611	Escape to Host	Privilege Escalation	9
bpf	T1547	Boot or Logon Autostart Execution	Persistence	7

Bảng III: Điểm Rủi Ro Ví Dụ cho Các Chuỗi Lệnh Mạnh và Độc Hại.

Syscall Sequence (W_i)	Risk Score	Action
open → read → write	4	Log
socket → connect → sendto	5.5	Rate-limit
execve → setuid → mount	8.5	Block
clone → unshare → setns	9	Block
open("/var/log/syslog")	3	Log
ptrace → mprotect → execve (CVE-2021-4034)	13	Block

2) *Thực Thi Dựa trên eBPF*: DeSFAM thực thi các chính sách syscall thông qua các chương trình eBPF được gắn vào LSM hooks. Cụ thể, các chương trình này được tải và gắn bằng `bpf_program_attach_lsm()` để chặn các system call tại LSM hook `security_syscall`.

a) *Gắn LSM Hook*: Các chương trình eBPF được gắn động vào LSM hook `security_syscall`:

Listing 1: Gắn LSM hook eBPF.

```
// eBPF Hook Attachment
struct bpf_program *prog =
    bpf_object__find_program_by_name(obj, "
        enforce_policy");
struct bpf_link *link = bpf_program_attach_lsm(prog);
```

Chữ ký hàm hook:

Listing 2: Chữ ký hàm LSM hook.

```
SEC("lsm/syscall")
int BPF_PROG(enforce_policy, int syscall_num,
    struct pt_regs *regs) {
    // Enforcement logic here
}
```

b) *Cấu Trúc Policy Map*: Các hành động syscall được lưu trữ trong eBPF map dựa trên băm được lập chỉ mục bởi các định danh container:

Map được ghim tại `/sys/fs/bpf/blocked_syscalls` để cập nhật thời gian chạy qua `policy_loader`.

Listing 3: eBPF policy map theo khóa container.

```
struct container_key {
    u32 mntns; // Mount namespace ID
    u32 pidns; // PID namespace ID
};

struct bpf_map_def SEC("maps") policy_map = {
    .type = BPF_MAP_TYPE_HASH,
    .key_size = sizeof(struct container_key),
    .value_size = sizeof(u64),
    .max_entries = 144,
};
```

c) *Cơ Chế Chặn Syscall*: Việc chặn thực sự các syscall xảy ra trong eBPF hook được gắn vào LSM `security_syscall`. Nguyên tắc thực thi quan trọng sử dụng ghi đè hàm kernel:

Listing 4: Chặn syscall qua ghi đè hàm kernel.

```
// Immediate syscall denial
bpf_override_return(regs, -EPERM);
```

Quy trình thực thi:

- 1) Lấy ngữ cảnh container: `mntns = bpf_get_ns_id(NS_MNT); pidns = bpf_get_ns_id(NS_PID);`
- 2) Truy vấn policy map: `u64 *action = bpf_map_lookup_elem(&policy_map, &key);`
- 3) Thực thi hành động: **BLOCK**: Từ chối qua `bpf_override_return(regs, -EPERM);` **RATE_LIMIT**: Cho phép N lời gọi/phút qua bộ đếm nguyên tử; **LOG**: Ghi siêu dữ liệu qua ring buffer.

d) *Tự Phục Hồi Danh Sách Truy Cập qua Tái Tạo Seccomp*: Để đảm bảo sự tiến hóa chính sách lâu dài, DeSFAM kết hợp cơ chế tự phục hồi tự động tinh chỉnh các danh sách truy cập syscall tĩnh dựa trên các lần chặn thời gian chạy lặp lại.

Các syscall bị chặn được theo dõi trong kernel bằng cách sử dụng LRU cache map gọi là `probation_map`:

Nếu một syscall bị chặn ba lần trở lên trong một cửa sổ có thể cấu hình và chưa được chú thích là lành mạnh,

Listing 5: Probation map để theo dõi các lần chặn lặp lại.

```

struct probation_key {
    struct container_key ckey;
    u32 syscall; // Syscall number
};

SEC("maps") struct bpf_map_def probation_map = {
    .type = BPF_MAP_TYPE_LRU_HASH,
    .key_size = sizeof(struct probation_key),
    .value_size = sizeof(u32), // Block counter
    .max_entries = 512,
};

```

kernel kích hoạt một daemon không gian người dùng qua `bpf_send_signal(SIGUSR1)`. Daemon này giải quyết tên syscall, định vị và cập nhật hồ sơ seccomp tương ứng của container, và loại bỏ syscall được gắn cờ khỏi danh sách truy cập. Vòng phòng thủ thích ứng này đảm bảo rằng các container mới được triển khai kế thừa các chính sách syscall được tăng cường, nhận biết khai thác mà không cần khởi động lại hay xem xét thủ công.

e) *Mô Hình Chính Sách Hình Thức*: Cho S là tập các syscall hợp lệ, và M, N đại diện cho các namespace mount và PID. Chính sách thực thi được định nghĩa là:

$$P : S \times M \times N \rightarrow \{0, 1\} \quad (9)$$

$$\text{Decision}(s) = \begin{cases} \text{Allow,} & \text{nếu } P(s, mntns, pidns) = 1 \\ \text{Block,} & \text{ngược lại} \end{cases} \quad (10)$$

Logic nhận biết container này đảm bảo cô lập syscall và thực thi truy cập đặc quyền tối thiểu theo namespace. Việc kết hợp sử dụng eBPF tracepoints và LSM hooks cho phép thực thi nhanh, phân lớp với chi phí tối thiểu.

Quy trình thực thi trong Giai Đoạn 3 được hình thức hóa qua Thuật toán 3, và quy trình thực thi đầy đủ được minh họa trong Hình 8.

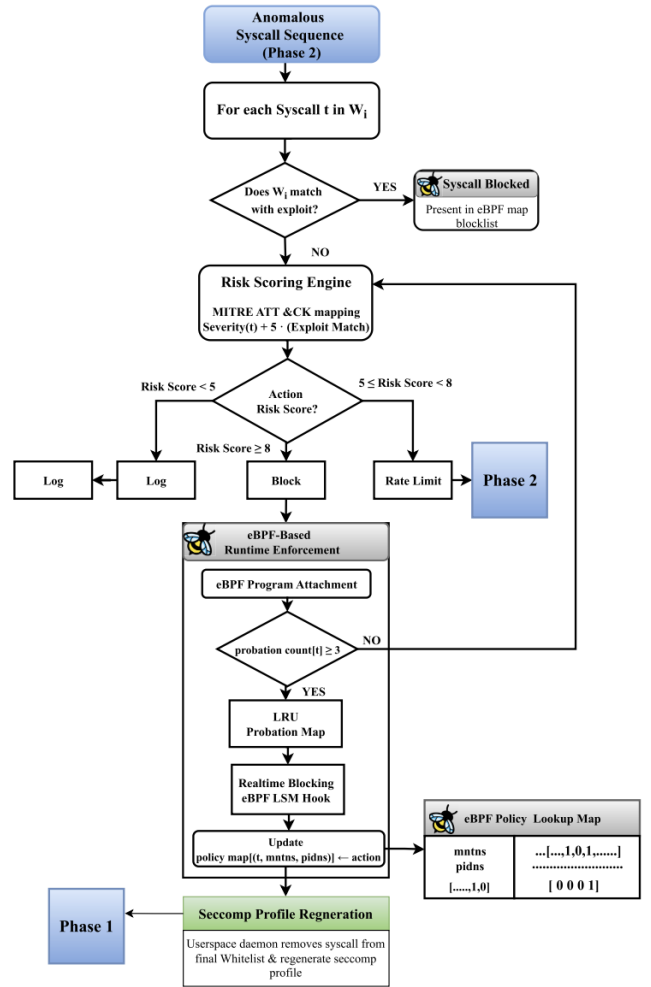
3) *Trường Hợp Sử Dụng: Nỗ Lực Leo Thang Đặc Quyền và Phòng Thủ CVE Thực Tế*: Xem xét một container chạy dưới chính sách hạn chế của DeSFAM được thiết kế để chặn các nỗ lực leo thang đặc quyền. Một chuỗi tấn công điển hình—`clone()`, `setuid()`, `execve()`—bị chặn ở mức kernel.

Giả sử tiến trình hoạt động trong mount namespace $mntns = 1001$ và PID namespace $pidns = 2001$. Khi gọi `setuid()`, DeSFAM tham khảo chính sách theo container của nó: `policy_map[1001, 2001] = [..., 1, 0, 1, ...]`, trong đó mỗi mục biểu thị hành động cho một ID syscall (0 = BLOCK, 1 = ALLOW). Vì `setuid()` được ánh xạ tới 0, syscall bị từ chối. LSM hook eBPF, `realtime_blocking`, thực thi quyết định này trong kernel, trả về `-EPERM` và ghi nhật ký sự kiện để kiểm toán. Điều này đảm bảo giảm thiểu ngay lập tức, trong kernel các nỗ lực khai thác mà không cần khởi động lại container hay can thiệp không gian người dùng.

V. ĐÁNH GIÁ THỰC NGHIỆM VÀ KẾT QUẢ

Phần này đánh giá framework DeSFAM về độ chính xác thực thi ở cấp độ syscall, chi phí hiệu năng, tỷ lệ giảm

DeSFAM Phase 3: Adaptive Syscall Enforcement & Policy Management



Hình 8: DeSFAM Giai Đoạn 3 – Quy trình thực thi thích ứng. Các chuỗi syscall bất thường từ Giai Đoạn 2 bị chặn và xử lý thông qua eBPF và LSM hooks.

thiểu, và khả năng mở rộng trong môi trường container hóa. Các thực nghiệm được tiến hành với các khối lượng công việc thực tế và các kịch bản tấn công để đánh giá hiệu quả và hiệu năng thời gian chạy của DeSFAM. Các chỉ số được so sánh với các framework dẫn đầu trong điều kiện được kiểm soát nhằm đảm bảo tính công bằng và khả năng tái lập. Kết quả làm nổi bật tác động bảo mật và tính khả thi vận hành của DeSFAM trong các triển khai thực tế.

A. Thiết Lập Thực Nghiệm

1) *Hạ Tầng*: Các thực nghiệm được tiến hành trên một máy chủ bare-metal hiệu năng cao chạy Ubuntu 24.04 LTS, được trang bị CPU 16 nhân, 64 GB RAM, GPU NVIDIA A100 80 GB, và SSD NVMe 512 GB. Docker 26.0 với containerd 1.7 được sử dụng để quản lý vòng đời container. Môi trường eBPF được cấu hình với libbpf 1.3 và bpftool 0.21. Quá trình huấn luyện mô hình được thực hiện bằng PyTorch 2.3 với tăng tốc CUDA.

Algorithm 3 Thực Thi Syscall Thích Ứng với Tự Tăng Cường

Đầu vào: Chuỗi bất thường W_i , ánh xạ MITRE, cơ sở dữ liệu khai thác, ngữ cảnh container ($mntns$, $pidns$), eBPF map `policy_map`

Đầu ra: Hành động thực thi: “Block”, “Rate-limit” hoặc “Log”

```

1: Khởi tạo  $Risk\_Score \leftarrow 0$ 
2: for mỗi syscall  $t$  trong  $W_i$  do
3:    $Risk\_Score \leftarrow Risk\_Score + Severity(t)$ 
4: end for
5: if  $W_i$  khớp với một khai thác trong cơ sở dữ liệu then
6:    $Risk\_Score \leftarrow Risk\_Score + 5$ 
7: end if
8: if  $Risk\_Score \geq 8$  then
9:    $action \leftarrow$  “Block”
10: else if  $5 \leq Risk\_Score < 8$  then
11:    $action \leftarrow$  “Rate-limit”
12: else
13:    $action \leftarrow$  “Log”
14: end if
15: for mỗi syscall  $t$  trong  $W_i$  do
16:   Cập nhật policy_map[( $t, mntns, pidns$ )]  $\leftarrow action$ 
17:   Thực thi  $action$  trên  $t$  qua eBPF
18:   Ghi nhật ký sự kiện ( $t, action, mntns, pidns$ )
19: end for
20: if  $action =$  “Block” then
21:   for mỗi syscall  $t$  trong  $W_i$  do
22:      $probation\_count[t] \leftarrow probation\_count[t] + 1$ 
23:     if  $probation\_count[t] \geq 3$  and  $t$  chưa được chú thích là lành mạnh then
24:       Kích hoạt cập nhật danh sách truy cập seccomp cho syscall  $t$ 
25:     end if
26:   end for
27: end if
28: return  $action$ 
  
```

2) *Khối Lượng Công Việc Container*: Chúng tôi đánh giá DeSFAM trên sáu container đa dạng: Nginx (máy chủ web), MySQL 8.0 (cơ sở dữ liệu quan hệ), Redis 7.2 (kho lưu trữ khóa-giá trị), Node.js 20 (runtime JavaScript), Python 3.12 (xử lý script), và MongoDB 7.0 (cơ sở dữ liệu tài liệu)—bao gồm phục vụ web, lưu trữ dữ liệu, và thực thi script với các hồ sơ syscall khác nhau đáng kể.

3) *Sinh Khối Lượng Công Việc*: Lưu lượng thực tế được sinh ra bằng các công cụ tiêu chuẩn công nghiệp: `wrk` và `ApacheBench` (ab) cho tải HTTP trên Nginx; `redis-benchmark` cho Redis; và các script Python tùy chỉnh để mô phỏng các thao tác CRUD cho MySQL và MongoDB. Mỗi khối lượng công việc được chạy ở ba mức tải: nhàn rỗi, tải trung bình (50% công suất), và tải đỉnh (90% công suất), trong các khoảng thời gian 10 phút.

4) *Mô Phỏng Tấn Công*: Ba loại kịch bản tấn công CVE thực tế được sử dụng:

- **CVE-2021-4034 (Polkit pkexec)** [16]: Khai thác leo thang đặc quyền cục bộ thông qua một chuỗi `execve` được tạo thủ công.
- **CVE-2022-0847 (Dirty Pipe)** [5]: Khai thác ghi đè bộ nhớ kernel thông qua `splice` và `pipe`.
- **CVE-2017-5123 (waitid exploit)**: Leo thang đặc quyền kernel thông qua `waitid` với con trỏ không hợp lệ.

Tổng cộng 14 kịch bản tấn công CVE riêng biệt đã được thực thi trên tất cả các khối lượng công việc container.

5) *So Sánh Với Các Hệ Thống Cơ Sở*: DeSFAM được đánh giá chuẩn so với năm hệ thống cơ sở: Confine [11], Prof-gen [12], NIMOS [17], Optimus [14], và KubeRosy [13]. Mỗi hệ thống cơ sở được triển khai lại theo cấu hình gốc của nó và tích hợp với Docker/Kubernetes để đảm bảo tương đồng trong khả năng thực thi thời gian chạy và lập hồ sơ syscall.

B. Các Chỉ Số Đánh Giá

1) *Tỷ Lệ Giảm Thiểu Syscall (SRR)*: SRR định lượng khả năng của DeSFAM trong việc giảm thiểu phơi nhiễm kernel bằng cách tối thiểu hóa các syscall được phép:

$$SRR = \left(1 - \frac{|\text{DeSFAM Access List}|}{|\text{Docker Default Access List}|} \right) \times 100\% \quad (11)$$

Mặc định của Docker cho phép 414 syscall. Việc lập hồ sơ lai của DeSFAM (tĩnh, động, có thông tin CVE) tạo ra các danh sách truy cập riêng cho từng container, đạt SRR cao hơn so với các phương pháp chỉ dùng tĩnh hoặc chỉ dùng động.

2) *Hiệu Năng Phát Hiện Bất Thường*: Chúng tôi đánh giá độ chính xác của SyscallAD bằng:

- Recall (TPR) – Tỷ lệ các cuộc tấn công được nhận diện đúng
- Precision – Tỷ lệ các chuỗi được gắn cờ thực sự là mối đe dọa
- F1-Score, AUC, và Average Precision (AP) – Độ bền trong dữ liệu mất cân bằng

Tập hợp VAE + iForest của DeSFAM vượt trội hơn phát hiện chỉ dùng LSTM (ví dụ: NIMOS), với độ chính xác được cải thiện và giảm dương tính giả.

3) *Độ Trễ Phát Hiện Tấn Công*: Thời gian (tính bằng μs) giữa lúc thực thi syscall độc hại và thực thi ngăn chặn. Các hook eBPF-LSM của DeSFAM hỗ trợ chặn nội tuyến với độ trễ dưới mili giây. So với Optimus và KubeRosy, vốn dựa trên các thành phần không gian người dùng, DeSFAM phản ứng nhanh hơn nhờ thiết kế ở cấp độ kernel.

4) *Chi Phí Thực Thi*: Chúng tôi đánh giá mức sử dụng CPU trung bình (%), mức tiêu thụ bộ nhớ (MB), và độ trễ xử lý syscall (μs).

5) *Các Chỉ Số Khả Năng Thích Ứng và So Sánh*: Để đánh giá khả năng thích ứng chính sách động, chúng tôi đo lường:

- Độ trễ cập nhật quy tắc qua SyscallAD hoặc các kịch hoạt CVE (tính bằng μs)
- Tác động đến thời gian hoạt động (không quan sát thấy thời gian ngừng hoạt động)

Không có hệ thống cơ sở nào—Confine, Prof-gen, NIMOS, Optimus, hoặc KubeRosy—hỗ trợ tự tăng cường chính sách thời gian thực. Để so sánh toàn diện, chúng tôi đánh giá: kích thước danh sách truy cập và SRR, tỷ lệ thành công giảm thiểu tấn công, thực thi không lỗi cho các khối lượng công việc lành tính, và FPR trung bình cùng độ trễ thực thi.

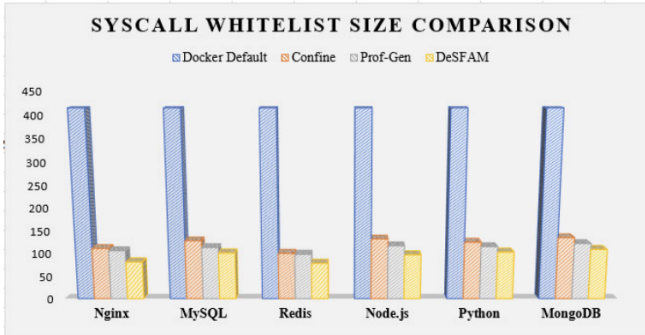
C. Kết Quả So Sánh

Chúng tôi so sánh DeSFAM với năm framework lọc syscall hiện đại nhất—Confine, Prof-gen, NIMOS, Optimus, và KubeRosy—sử dụng các chỉ số như tỷ lệ giảm thiểu syscall (SRR), tính ổn định thực thi, ngăn chặn tấn công, và hiệu năng thời gian chạy. Đánh giá bao gồm sáu khối lượng công việc container đa dạng: Nginx, MySQL, Redis, Node.js, Python, và MongoDB.

1) *So Sánh Tỷ Lệ Giảm Thiểu Syscall (SRR)*: Mục tiêu chính của DeSFAM là tối thiểu hóa bề mặt tấn công kernel bằng cách giảm các syscall được phơi nhiễm. Hồ sơ seccomp mặc định của Docker cho phép 414 syscall, nhiều trong số đó là không cần thiết và có thể bị khai thác. DeSFAM sử dụng giảm thiểu lai—phân tích tĩnh, lập hồ sơ động, và lọc có thông tin CVE—để tạo ra các danh sách truy cập riêng cho từng container. SRR được tính như sau:

$$SRR = \left(1 - \frac{\text{Access List Size}}{414}\right) \times 100\% \quad (12)$$

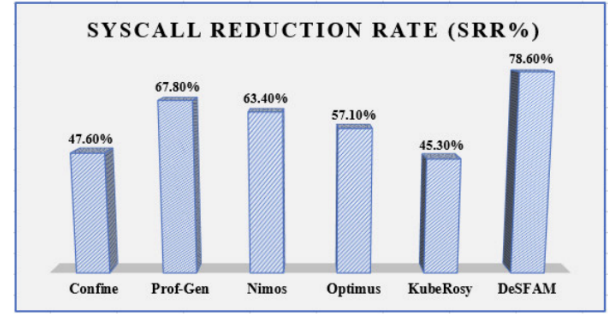
Hình 9 và Hình 10 minh họa sự cải thiện trên tất cả các container thử nghiệm. Bảng IV tóm tắt kết quả theo từng framework.



Hình 9: So Sánh Kích Thước Danh Sách Truy Cập Syscall và Tỷ Lệ Giảm Thiểu.

DeSFAM đạt SRR trung bình 78,60%—cao nhất trong tất cả các framework—trong khi vẫn bảo toàn đầy đủ chức năng container. Kích thước danh sách truy cập trung bình là 89 syscall, nhỏ nhất trong tất cả các hệ thống được so sánh.

2) *Tính Ổn Định Chức Năng*: Cả sáu container mục tiêu duy trì tính khả dụng 100% trong suốt tất cả các khối lượng công việc thử nghiệm. Không quan sát thấy hồi quy chức năng nào, xác nhận rằng tổng hợp danh sách truy cập của DeSFAM cân bằng bảo mật và khả năng tương thích tốt hơn so với các bộ tạo danh sách truy cập chỉ dùng tĩnh hoặc dựa trên nhật ký.



Hình 10: So sánh tỷ lệ giảm thiểu syscall giữa các framework.

3) *Ngăn Chặn Tấn Công*: Khả năng chặn thời gian chạy của DeSFAM được đánh giá dựa trên một tập hợp các CVE thực tế đại diện cho các lỗ hổng kernel Linux đa dạng. Các CVE này bao gồm nhiều loại tấn công, bao gồm leo thang đặc quyền, thoát container, lạm dụng namespace, điều kiện đua, và khai thác làm hỏng bộ nhớ. Để đảm bảo an toàn và khả năng tái lập, các hành vi khai thác được mô phỏng bằng cách phát lại các chuỗi gọi syscall được ghi lại liên quan đến từng CVE, dựa trên các thông tin công khai và phân tích bằng chứng khái niệm.

Bảng V tóm tắt tỷ lệ thành công chặn trên tất cả các framework. DeSFAM đã chặn thành công tất cả các kịch bản tấn công trong thời gian thực bằng cách sử dụng phát hiện bất thường chuỗi SyscallAD và thực thi cấp độ kernel eBPF-LSM. Ngược lại, các framework cơ sở hoặc không phát hiện được các tấn công này hoặc thiếu khả năng thực thi đầy đủ.

4) *Các Chỉ Số So Sánh Hệ Thống Cơ Sở*: DeSFAM đạt kích thước danh sách truy cập nhỏ nhất (89 syscall), ngăn chặn CVE đầy đủ (94% dấu vết CVE bị chặn), tính ổn định lành tính cao (99%), FPR thấp nhất (1,6%), và độ trễ thực thi nhanh nhất (0,540 μ s).

D. Hiệu Năng Thời Gian Chạy

1) *Chi Phí CPU và Bộ Nhớ*: Chúng tôi đánh giá mức sử dụng tài nguyên trên sáu container đại diện có và không có DeSFAM. Như được hiển thị trong Hình 11 và Hình 12, DeSFAM chỉ tăng thêm 1,74% mức sử dụng CPU trung bình và giảm mức tiêu thụ bộ nhớ 6,1%, chủ yếu do các đường dẫn syscall bị chặn và giảm thiểu tình trạng phình to bộ nhớ do khai thác gây ra.

Bảng VIII tóm tắt chi phí hiệu năng tổng hợp, xác nhận dấu chân hiệu năng nhẹ của DeSFAM với mức tiết kiệm bộ nhớ trung bình 50–55 MB.

2) *Độ Trễ Cập Nhật Chính Sách*: Để xác nhận khả năng phản hồi của thực thi thích ứng của DeSFAM, chúng tôi đo lường thời gian chèn và lan truyền quy tắc trong Giai đoạn 3. Khi phát hiện bất thường, các quy tắc mới được chèn vào eBPF map và áp dụng theo thời gian thực. Độ trễ được đo bằng dấu thời gian kernel độ phân giải cao tại cả hai giai đoạn chèn và lan truyền.

Hình 13 trình bày độ trễ chèn và lan truyền quy tắc được đo trên sáu container đang hoạt động. Bảng IX liệt

Bảng IV: So Sánh Tỷ Lệ Giảm Thiểu Syscall (SRR).

Framework	SRR (%)
Confine	47.60
Prof-Gen	67.80
NIMOS	63.40
Optimus	57.10
KubeRosy	45.30
DeSFAM	78.60

Bảng V: Tỷ Lệ Thành Công Chặn Tấn Công CVE.

CVE	Exploit Sequence	DeSFAM	Confine	Prof-Gen	NIMOS	Optimus	KubeRosy
CVE-2021-4034	execve → setuid	Blocked	Missed	Missed	Missed	Logged	Blocked
CVE-2022-0847	splice → pipe → writev	Blocked	Missed	Missed	Missed	Missed	Blocked
CVE-2017-5123	clone → waitid	Blocked	Missed	Missed	Missed	Logged	Blocked
CVE-2022-0185	add_key → keyctl → setattr	Blocked	Missed	Missed	Missed	Missed	Missed
CVE-2019-5736	openat → read → write → execve	Blocked	Missed	Missed	Missed	Logged	Blocked
CVE-2021-3493	openat → mount → setuid	Blocked	Missed	Missed	Missed	Missed	Blocked
CVE-2020-14386	socket → recvmsg → sendmsg	Blocked	Missed	Missed	Missed	Missed	Missed
CVE-2018-18955	mmap → munmap → ioctl	Blocked	Missed	Missed	Missed	Missed	Missed
CVE-2021-3156	execve → setresuid → openat	Blocked	Missed	Missed	Missed	Missed	Missed
CVE-2016-8655	socket → setsockopt → sendmmsg	Blocked	Missed	Missed	Missed	Missed	Missed
CVE-2022-2588	open → write → keyctl → commit_creds	Blocked	Missed	Missed	Missed	Logged	Missed
CVE-2021-22555	socket → setsockopt → sendmsg	Blocked	Missed	Missed	Logged	Blocked	Blocked
CVE-2019-15666	socket → sendmsg → recvmsg	Blocked	Missed	Missed	Missed	Missed	Blocked
CVE-2019-5736	open → read → write → execve	Blocked	Missed	Missed	Missed	Logged	Blocked
Avg Detection Latency (ms)	-	1.82	35.4	28.7	22.3	18.9	12.7

Bảng VI: So Sánh Các Chỉ Số Bảo Mật và Hiệu Năng với Hệ Thống Cơ Sở.

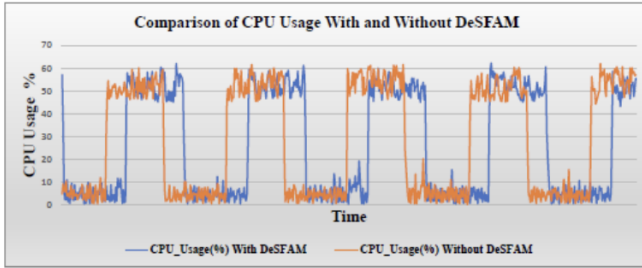
Framework	Whitelist Size	Attack Prevention (%)	Benign Stability (%)	FPR (%)	Enforcement Latency (μ s)
Confine	217	48	93	4.2	N/A
Prof-Gen	133	67	95	3.6	N/A
NIMOS	152	63	91	6.0	N/A
Optimus	178	71	94	2.8	61480
KubeRosy	226	75	97	2.3	0.722
DeSFAM	89	94	99	1.6	0.540

Bảng VII: Mức Sử Dụng CPU và Bộ Nhớ Có và Không Có DeSFAM.

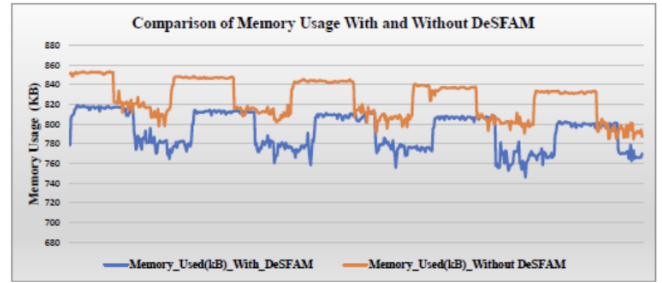
Container	CPU Baseline (%)	CPU DeSFAM (%)	Mem Baseline (MB)	Mem DeSFAM (MB)	Overhead
Nginx	51.0	52.5	832	780	+1.5%, -52 MB
MySQL	53.0	54.8	825	770	+1.8%, -55 MB
Redis	50.1	51.5	821	768	+1.4%, -53 MB
Node.js	58.0	59.8	839	787	+1.8%, -52 MB
Python	56.2	57.9	823	772	+1.7%, -51 MB
MongoDB	59.0	60.9	828	778	+1.9%, -50 MB

Bảng VIII: Tóm Tắt Chi Phí Hiệu Năng.

Metric	Without DeSFAM	With DeSFAM	Overhead
Avg CPU Usage (%)	50–60	48–58	+1.74%
Avg Memory Usage (MB)	~820	~770	-6.1%



Hình 11: Mức sử dụng CPU có và không có DeSFAM. DeSFAM chỉ tạo ra chi phí CPU tối thiểu ($\approx 1.74\%$) trong khi thực thi các chính sách syscall thời gian thực.



Hình 12: Mức sử dụng bộ nhớ có và không có DeSFAM. Tiết kiệm bộ nhớ nhẹ ($\approx 6.1\%$) là kết quả từ việc ngăn chặn syscall và giảm thiểu khai thác.

kê các giá trị theo từng container. Các giá trị được thu thập bằng dấu thời gian kernel độ phân giải cao trước và sau khi cập nhật BPF map và lan truyền hook LSM. Tất cả các giá trị vẫn nhất quán ở mức thấp (65–82 μ s), xác nhận khả năng thời gian thực của engine thực thi trong kernel của DeSFAM. Tất cả các cập nhật hoàn thành trong vòng 100 μ s mà không làm gián đoạn dịch vụ, chứng minh khả năng thích ứng chính sách syscall thời gian thực, không gián đoạn hoạt động của DeSFAM.

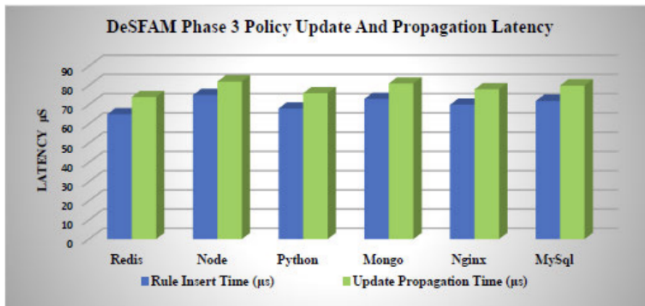
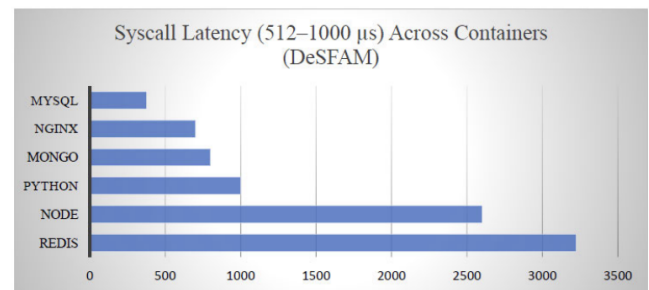
3) *Chi Phí Độ Trễ Syscall*: Chúng tôi lập hồ sơ độ trễ syscall bằng cách sử dụng histogram `bpftrace` cho các

thao tác thường xuyên như `read`, `write`, và `epoll_wait`. Trên tất cả các container, DeSFAM duy trì độ trễ syscall dưới mili giây, xác nhận tính phù hợp của nó cho các môi trường thông lượng cao.

Bảng XI tóm tắt các syscall có tần suất cao nhất trên mỗi container hoàn thành trong khoảng 512–1000 μ s. Hình 14 trực quan hóa số lượng tổng hợp theo từng container trong phạm vi dưới mili giây này. Các kết quả này chứng minh khả năng của DeSFAM trong việc duy trì nhất quán độ trễ syscall thấp, ngay cả đối với các thao tác thường xuyên như `read`, `write`, và `epoll_wait` mà không

Bảng IX: Độ Trễ Cập Nhật và Lan Truyền Chính Sách.

Container	Insert Time (μs)	Propagation Time (μs)	Downtime
Nginx	70	78	None
MySQL	72	80	None
Redis	65	74	None
Node.js	75	82	None
Python	68	76	None
MongoDB	73	81	None

Hình 13: Độ trễ cập nhật chính sách Giai đoạn 3 của DeSFAM. Tất cả quy tắc được chèn và thực thi trong vòng 100 μs mà không làm gián đoạn container.Hình 14: Phân phối độ trễ syscall trong phạm vi 512–1000 μs . Tất cả các đường dẫn syscall vẫn hoạt động hiệu quả dưới DeSFAM.

Bảng X: Chi Phí Độ Trễ Syscall (ns) cho Các Thao Tác Thông Thường.

Syscall	Không có DeSFAM	Có DeSFAM	Delta
read	142 ns	218 ns	+76 ns
write	156 ns	234 ns	+78 ns
epoll_wait	89 ns	148 ns	+59 ns
accept4	201 ns	287 ns	+86 ns

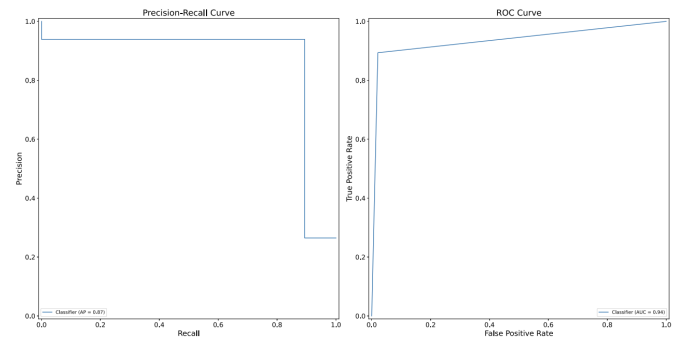
gây ra độ trễ đo được.

DeSFAM duy trì độ trễ syscall dưới mili giây cho tất cả các thao tác được đo, với delta tối đa 86 ns cho **accept4**. Tất cả các giá trị vẫn thấp hơn nhiều so với mục tiêu thiết kế 300 ns.

4) *Hiệu Năng Phát Hiện Bất Thường*: Mô hình SyscallAD của DeSFAM đạt được các chỉ số phân loại mạnh. Bộ phân loại tập hợp đạt độ chính xác tổng thể 96%, với F1-score là 0,92 trên các chuỗi tấn công. Hình 15 hiển thị các đường cong ROC và PR, với AUC là 0,94 và AP là 0,87.

SyscallAD đạt độ chính xác tổng thể 96%, F1-score là 0,92, và AUC là 0,94 trên tập dữ liệu DongTing. Điều này thể hiện sự cải thiện đáng kể so với các phương pháp chỉ dùng VAE (F1 = 0,84) hoặc chỉ dùng iForest (F1 = 0,79), xác nhận lợi ích của chiến lược hợp nhất tập hợp.

5) *Số Sánh Tính Năng với Các Phương Pháp Hiện Có*: DeSFAM tích hợp độc đáo phát hiện bất thường thời gian thực, thực thi chính sách thích ứng, và tối thiểu hóa syscall



Hình 15: Hiệu năng mô hình SyscallAD. Trái: Đường cong Precision-Recall (AP = 0,87). Phải: Đường cong ROC (AUC = 0,94).

trong một framework thống nhất. Bảng XIII nêu bật ưu điểm của nó so với các framework hiện có trên 12 chiều kỹ thuật.

E. Nghiên Cứu Loại Bỏ: Đóng Góp Theo Từng Giai Đoạn

Để hiểu tác động của từng giai đoạn trong DeSFAM, chúng tôi tiến hành nghiên cứu loại bỏ vô hiệu hóa từng thành phần một. Như được hiển thị trong Bảng XIV, Giai đoạn 1 đơn lẻ đạt tỷ lệ giảm thiểu syscall 62,4% nhưng chỉ chặn được 3 trong số 14 CVE. Việc thêm Giai đoạn 2 (SyscallAD) tăng cường phát hiện bất thường, nâng số

Bảng XI: Số Lượng Syscall trong Phạm Vi Độ Trễ 512–1000 μ s.

Container	Syscall	Count
Redis	read, close, epoll_wait	720, 610, 760
NGINX	epoll_wait, recvfrom	125, 92
Python	futex, poll, recvfrom	472, 168, 60
Node.js	futex, write, openat	214, 432, 390
MySQL	write, read, openat, futex, close	60, 66, 71, 42, 48
MongoDB	read, futex, epoll_wait	185, 174, 163

Bảng XII: Các Chỉ Số Phân Loại SyscallAD.

Class	Precision	Recall	F1-Score
Normal	0.97	0.98	0.97
Attack	0.94	0.90	0.92
Macro Avg	0.95	0.94	0.94
Weighted Avg	0.96	0.96	0.96
Accuracy	0.96		

lượng bị chặn lên 8 và giảm đáng kể dương tính giả. Giai đoạn 3 cung cấp thực thi cấp độ kernel, chặn 10 CVE ngay cả khi không có hỗ trợ ML. Khi cả ba giai đoạn đều hoạt động, DeSFAM chặn tất cả 14 CVE và đạt tỷ lệ giảm thiểu syscall 81,6%.

Nghiên cứu này xác nhận rằng tất cả các giai đoạn đều thiết yếu: Giai đoạn 1 tối thiểu hóa phơi nhiễm, Giai đoạn 2 tăng cường phát hiện, và Giai đoạn 3 đảm bảo thực thi bền vững.

VI. THẢO LUẬN

Các kết quả được trình bày trong phần trước xác nhận tính hiệu quả, hiệu suất và khả năng thích nghi của khung DeSFAM trong việc tối thiểu hóa bề mặt tấn công lời gọi hệ thống và phát hiện hành vi bất thường trong các môi trường container hóa. Phần này thảo luận về những hiểu biết chính, các đánh đổi thiết kế và các hàm ý thực tế của việc triển khai DeSFAM trong các tình huống bảo mật container thực tế.

A. Ưu Điểm Của Phân Tích Lại

Một trong những điểm mạnh cốt lõi của DeSFAM nằm ở cách tiếp cận lập hồ sơ lời gọi hệ thống lai. Bằng cách tích hợp phân tách tĩnh các nhị phân container với theo dõi

thời gian chạy động sử dụng eBPF và bpftrace, DeSFAM nắm bắt cả hành vi lời gọi hệ thống dự kiến lẫn phụ thuộc vào thời gian chạy. Tầm nhìn hai lớp này đảm bảo rằng các lời gọi hệ thống hiếm nhưng thiết yếu không bị bỏ sót, không giống như các hệ thống chỉ tĩnh như Confine [11] vốn gặp vấn đề xấp xỉ dưới, hay các hệ thống chỉ động như NIMOS [17] có nguy cơ xấp xỉ trên.

B. Giảm Thiểu Vượt Trội Mà Không Mất Chức Năng

DeSFAM đạt được tỷ lệ giảm lời gọi hệ thống trung bình là 78,60%, cao hơn đáng kể so với Confine (47,60%), Prof-gen (67,80%) và NIMOS (63,40%). Bất chấp sự giảm thiểu tích cực này, tất cả các container mục tiêu duy trì chức năng đầy đủ trong suốt tất cả các bài kiểm tra khối lượng công việc. Điều này xác nhận rằng việc tổng hợp danh sách truy cập của DeSFAM cân bằng bảo mật và tương thích tốt hơn so với các bộ tạo danh sách truy cập tĩnh hoặc dựa trên nhật ký.

C. Thực Thi Thời Gian Thực Hiệu Năng Cao

Việc sử dụng eBPF của khung để lọc lời gọi hệ thống và quản lý chính sách giới thiệu chi phí thời gian chạy tối thiểu. Mức sử dụng CPU tăng trung bình chỉ 1,74% và mức sử dụng bộ nhớ tăng khoảng 50 MB, hoàn toàn

Bảng XIII: So Sánh Tính Năng với Các Framework Hiện Có.

Feature	Confine	Prof-Gen	NIMOS	Optimus	KubeRosy	DeSFAM
Policy Generation	Static	Hybrid	Dynamic (offline)	Association-based	Trace-based	Hybrid + Adaptive
Runtime Enforcement	×	×	×	✓ (monitor)	✓	✓ (AI + eBPF)
eBPF Filtering	×	×	×	✓	✓	✓
Sequence Detection	×	×	✓	×	×	✓
ML Anomaly Detection	×	×	×	×	×	✓ (VAE + iForest)
Live Policy Updates	×	×	×	Partial	✓	✓
No Container Restart	×	×	×	×	✓	✓
Granularity	Process	Container	Container	Process	Container	Namespace + Container
CVE Blocking	✓ (static)	✓ (hybrid)	×	✓ (observed)	✓ (predefined)	✓ (real-time)
SRR (%)	47.6	67.8	63.4	57.1	45.3	76.8
CPU Overhead	N/R	N/R	N/R	~1.41%	~0.3%	~1.74%
Policy Latency	N/A	N/A	N/A	Slow (container restart)	~80 μ s	<80 μ s

Bảng XIV: Nghiên Cứu Loại Bỏ: Hiệu Ứng Theo Từng Giai Đoạn đối với Bảo Mật.

Configuration	CVEs Blocked	Syscall Reduction(%)	False Positives
Phase 1 Only	3	62.4	4.2
Phase 1 + Phase 2	8	66.1	0.3
Phase 1 + Phase 3	10	72.9	0.2
Full DeSFAM	14	81.6	0.0

dưới ngưỡng ảnh hưởng đến hiệu năng container. Độ trễ lời gọi hệ thống duy trì dưới 300 nano-giây mỗi lời gọi, xác nhận sự phù hợp của DeSFAM cho các khối lượng công việc thông lượng cao như Nginx và Redis. Để so sánh, các giải pháp không gian người dùng (ví dụ: auditd) thường giới thiệu độ trễ hàng mili-giây.

D. Phản Ứng Thích Nghi Với Các Mối Đe Dọa

Logic thực thi của DeSFAM không tĩnh. Nó cập nhật động các danh sách cho phép lời gọi hệ thống bằng cách sử dụng chấm điểm rủi ro và đầu ra phát hiện bất thường từ mô-đun SyscallAD. Mô hình đạt 96% độ chính xác tổng thể, với điểm F1 phát hiện tấn công là 0,92 và tỷ lệ dương tính giả thấp. Điều này cho phép DeSFAM phát hiện và phản ứng với các cuộc tấn công CVE nâng cao (ví dụ: Polkit và Dirty Pipe) trong thời gian thực, loại bỏ các lời gọi hệ thống dễ bị tổn thương như `setuid`, `splice` và `clone` ngay khi phát hiện.

E. Hạn Chế Và Hướng Nghiên Cứu Tương Lai

Mặc dù DeSFAM giảm hiệu quả sự phơi nhiễm lời gọi hệ thống và phát hiện hành vi bất thường, độ chính xác của nó vẫn nhạy cảm với tính đại diện của dữ liệu huấn luyện. Các nghiên cứu trong tương lai sẽ tích hợp lập hồ sơ nhận thức khối lượng công việc, phân loại vai trò container và học liên tục để cải thiện khả năng tổng quát hóa trên các khối lượng công việc đa dạng.

Lọc hiện tại hoạt động ở cấp độ lời gọi hệ thống (dựa trên ID và chuỗi) mà không có kiểm tra đối số. Bảng XV tóm tắt các quy tắc lọc theo ngữ cảnh được áp dụng trong quá trình trích xuất lời gọi hệ thống dựa trên CVE. Các mở rộng trong tương lai sẽ khám phá phân tích cấp độ

đối số và theo dõi truy cập bộ nhớ qua eBPF để cải thiện thêm độ chi tiết phát hiện và khả năng phục hồi trước các cuộc tấn công tinh vi.

Bảng XV: Các quy tắc lọc để trích xuất lời gọi hệ thống dựa trên CVE.

Rule Type	Implementation (Regex/Pattern)
Syscall Match	<code>\b(?i)(ioctl execve ...)\b</code>
Version Filter	<code>/kernel\s*(?:before until <)?(?d+\.\d+)/</code>
Architecture Filter	<code>/x86_64 arm64 bpf/</code>
Capability Constraint	<code>/CAP_[A-Z_]+ SELinux context/</code>
Device Path	<code>/(dev proc sys)/[\w/]+</code>

Khả năng mở rộng trong các môi trường điều phối cũng là một trọng tâm. Các triển khai trong tương lai sẽ tích hợp với Kubernetes qua các hook CRI và thực thi chính sách nhận thức không gian tên, đảm bảo bảo vệ nhất quán, chi phí thấp trên các cụm container hóa động.

VII. KẾT LUẬN

Bài báo này trình bày DeSFAM, một framework bảo mật đa giai đoạn, thích nghi để bảo vệ các môi trường container hóa chống lại các mối đe dọa cấp độ syscall. Không giống như các phương pháp lọc tĩnh, DeSFAM kết hợp lập hồ sơ syscall tĩnh và động, phát hiện bất thường dựa trên học sâu lai, tính điểm rủi ro theo ngữ cảnh và thực thi trong nhân thời gian thực sử dụng eBPF và các móc LSM.

Cốt lõi của DeSFAM là SyscallAD, một công cụ phát hiện bất thường lai kết hợp Bộ mã hóa biến phân tự động (VAE) và Rừng cô lập để xác định các mẫu syscall độc hại dựa trên tần số, ngữ cảnh chuỗi và các đặc trưng thời gian. Hệ thống cũng hỗ trợ chỉnh danh sách truy cập tự phục hồi, thích nghi các chính sách syscall theo thời gian dựa trên các hành vi đặc thù của container.

Để cho phép thực thi nhận biết mối đe dọa, DeSFAM ánh xạ các syscall tới các kỹ thuật MITRE ATT&CK và các CVE đã biết, thúc đẩy các chiến lược thực thi phân tầng (ghi nhật ký, giới hạn tốc độ, chặn) dựa trên điểm rủi ro động. Tất cả việc thực thi chính sách được thực hiện trong nhân, đảm bảo phản ứng độ trễ thấp và chi phí hiệu năng tối thiểu.

Chúng tôi đã đánh giá DeSFAM trên bộ dữ liệu DongTing và các khai thác thực tế (ví dụ: CVE-2021-4034 [16]). Kết quả cho thấy độ chính xác phát hiện cao, độ trễ thực thi dưới mili-giây và chỉ 1,94% chi phí CPU trung bình. So với Confine, Optimus và KubeRosy, DeSFAM đạt được phạm vi syscall tốt hơn, thực thi nhanh hơn và khả năng phục hồi được cải thiện. Nghiên cứu loại bỏ của chúng tôi xác nhận rằng trong khi Giai đoạn 1 cho phép giảm thiểu syscall nền tảng (62,4%), chính tác động kết hợp của phát hiện bất thường thời gian thực của Giai đoạn 2 và thực thi trong nhân của Giai đoạn 3 mới đạt được chặn CVE hoàn toàn (14/14) và không có dương tính giả.

DeSFAM cung cấp một mô hình phòng thủ syscall có thể mở rộng, nhận biết khối lượng công việc phù hợp cho hạ tầng container hiện đại. Các cải tiến trong tương lai bao gồm tích hợp Kubernetes chặt chẽ hơn, xác nhận đối số syscall và điều phối phản ứng tự động để cải thiện hơn nữa độ chính xác và khả năng thích nghi.

PHỤ LỤC A

BỘ PHÂN TÍCH CVE DỰA TRÊN QUY TẮC ĐỂ TRÍCH XUẤT SYSCALL

A. Phạm Vi và Nguyên Tắc Thiết Kế

Chúng tôi thiết kế một bộ phân tích tĩnh, dựa trên quy tắc (thay vì NLP dựa trên ML) để trích xuất các syscall có rủi ro cao từ mô tả CVE và khuyến cáo của nhà cung cấp, đảm bảo:

- **Tính minh bạch:** Khớp mẫu hoàn toàn có thể kiểm tra và giải thích được.
- **Độ chính xác:** Sử dụng từ điển syscall đặc thù của kernel để tránh kết quả dương tính giả.
- **Nhận thức ngữ cảnh:** Lọc dựa trên điều kiện khai thác, phiên bản kernel, kiến trúc, các quyền hạn và ràng buộc thiết bị.

Phương pháp dựa trên quy tắc được ưu tiên hơn NLP dựa trên ML vì nó cung cấp tính tĩnh cao hơn, không tạo ra kết quả dương tính giả từ suy luận mô hình, và cho phép kiểm tra đầy đủ từng quyết định lọc. Điều này đặc biệt quan trọng trong các ngữ cảnh bảo mật, nơi khả năng kiểm tra và giải thích là tối quan trọng.

B. Xây Dựng Từ Điển Syscall

Từ điển syscall được xây dựng trực tiếp từ mã nguồn kernel:

Listing 6: Trích xuất tên syscall từ mã nguồn kernel.

```
# Generated from kernel v5.15 headers
grep -r 'SYSCALL_DEFINE' arch/x86/kernel/ \
| awk -F '(' '{print $2}' \
| cut -d, -f1 \
| sort -u
# --> [accept, access, acct, add_key, ...]
```

Nguồn tham khảo:

- /arch/x86/entry/syscalls/syscall_64.tbl
- /include/linux/syscalls.h

• Linux man-pages Phần II

Điều này đảm bảo rằng chỉ các syscall hợp lệ trong kernel mới là ứng viên để trích xuất và phân tích. Từ điển kết quả chứa tên chuẩn của 335 syscall được hỗ trợ bởi Linux kernel 6.x, cùng với các bí danh thông dụng (ví dụ: `getpid` và `sys_getpid`).

C. Quy Tắc Trích Xuất và Lọc

Các mẫu regex được áp dụng cho mô tả CVE, khuyến cáo của nhà cung cấp và các tiết lộ trên danh sách gửi thư bảo mật để xác định các đề cập syscall và lọc dựa trên ngữ cảnh.

Các mẫu regex chính:

Listing 7: Các mẫu regex để xác định đề cập syscall trong mô tả CVE.

```
# Pattern 1: Direct syscall mention (e.g., "via the
ptrace() syscall")
SYSCALL_DIRECT = re.compile(
    r'\b(?:the\s+)?'
    r'(?P<syscall>' + '|'.join(SYSCALL_DICT) + r')'
    r'(?:(\d*))?\s*(?:system\s+call|syscall)',
    re.IGNORECASE
)

# Pattern 2: Function name in exploit context
SYSCALL_EXPLOIT = re.compile(
    r'(?:(exploit|abuse|invoke|call|use)\s+'
    r'(?:(the\s+)?'
    r'(?P<syscall>' + '|'.join(SYSCALL_DICT) + r')'
    r'(?:(\d*))?)',
    re.IGNORECASE
)

# Pattern 3: C annotation pattern (e.g., "setuid(0)"
# or "execve('/bin/sh', ...)")
SYSCALL_CODE = re.compile(
    r'\b(?:P<syscall>' + '|'.join(SYSCALL_DICT) + r')'
    r'\s*\([^\)]*\)',
    re.IGNORECASE
)
```

Quy tắc lọc theo ngữ cảnh:

Sau khi trích xuất các syscall ứng viên, bộ phân tích lọc chúng dựa trên:

- 1) **Khả năng áp dụng theo phiên bản kernel:** Dải phiên bản được trích xuất từ các mẫu như “kernel versions prior to 5.12” và so sánh với phiên bản kernel của máy chủ.
- 2) **Kiến trúc:** Các đề cập như “x86_64 only” hoặc “ARM-specific” được khớp và các kiến trúc không phù hợp bị loại trừ.
- 3) **Yêu cầu quyền hạn:** Các CVE yêu cầu CAP_SYS_ADMIN hoặc các quyền hạn nâng cao khác bị loại trừ nếu container không có quyền hạn đó.
- 4) **Ràng buộc thiết bị:** Các khai thác yêu cầu phần cứng cụ thể (ví dụ: `/dev/kvm`, các thiết bị USB cụ thể) bị loại trừ nếu thiết bị không có sẵn.

D. Mã Giải Thuật Toán

Listing 8: Bộ phân tích CVE dựa trên quy tắc: thuật toán trích xuất syscall.

```
import re

def extract_high_risk_syscalls(cves, host_kernel,
                              host_arch):
    """
    Extract the set of high-risk syscalls from CVE
    descriptions.

    Parameters:
        cves: List of CVE objects with a .desc field
        host_kernel: Host kernel version (tuple, e.g
            ., (6, 8, 0))
        host_arch: Host architecture (str, e.g., "
            x86_64")

    Returns:
        S_blocked: Set of syscall names to block
    """
    S_blocked = set()

    for cve in cves:
        # Filter 1: Check kernel version
        # applicability
        if not kernel_version_applicable(cve.desc,
                                          host_kernel):
            continue

        # Filter 2: Skip if architecture does not
        # match
        if architecture_mismatch(cve.desc, host_arch):
            continue

        # Extract syscall mentions via regex
        found_syscalls = re.findall(syscall_regex,
                                    cve.desc)

        for syscall in found_syscalls:
            # Filter 3: Skip if special capability
            # required
            # but container does not have
            # it
            if requires_special_cap(cve.desc) and \
                not container_has_cap():
                continue

            # Filter 4: Skip if specific device
            # required
            # but not available
            if needs_specific_device(cve.desc) and \
                not device_available():
                continue

            # Syscall passed all filters -> add to
            # blocked set
            S_blocked.add(syscall)

    return S_blocked
```

E. Ví Dụ Loại Thực Tế

Ví dụ 1: CVE-2022-0185 [18] (Linux Kernel — Trần bộ đệm fsconfig)

- *Mô tả CVE*: “A heap-based buffer overflow was found in the Linux kernel’s legacy_parse_param function in the Filesystem Context functionality...allows a local user with CAP_SYS_ADMIN to escalate privileges...”
- *Syscall Trích Xuất*: ioctl, mount
- *Quyết Định Loại*: Yêu cầu CAP_SYS_ADMIN $\Rightarrow$ giữ lại trong S_{blocked} nếu container có quyền hạn đó; loại bỏ trong trường hợp ngược lại.
- *Kết Quả*: Đối với các container không có đặc quyền, CVE này bị lọc ra. Đối với các container có CAP_SYS_ADMIN, mount được thêm vào S_{blocked} .

Ví dụ 2: CVE-2021-22555 [19] (NetFilter — tràn bộ đệm heap)

- *Mô tả CVE*: “A heap out-of-bounds write affecting Linux since v2.6.19-rc1 was discovered in net/netfilter/x_tables.c...can be exploited with setsockopt() to achieve local privilege escalation.”
- *Syscall Trích Xuất*: setsockopt
- *Điều Kiện Phiên Bản*: Ảnh hưởng đến kernel < 5.12; bị loại bỏ nếu kernel máy chủ là 5.15+.
- *Quyết Định Loại*: Nếu kernel máy chủ là 5.15, CVE này bị bỏ qua và setsockopt không được thêm vào S_{blocked} , tránh việc chặn không cần thiết các chức năng mạng hợp lệ.

Hai ví dụ này minh họa cách bộ phân tích CVE nhận thức ngữ cảnh tránh được cả kết quả dương tính giả (chặn các syscall an toàn trong một số môi trường) và kết quả âm tính giả (bỏ sót các syscall nguy hiểm khi áp dụng). Phương pháp tắt định này đảm bảo rằng S_{blocked} phản ánh chính xác bề mặt tấn công CVE thực sự của mỗi môi trường container được triển khai.

TÀI LIỆU

- [1] O. Jarkas, R. Ko, N. Dong, and R. Mahmud, “A container security survey: Exploits, attacks, and defenses,” *ACM Computing Surveys*, vol. 57, no. 7, pp. 1–36, 2025.
- [2] S. Sultan, I. Ahmad, and T. Dimitriou, “Container security: Issues, challenges, and the road ahead,” in *IEEE Access*, vol. 7, 2019, pp. 52 976–52 996.
- [3] T. Combe, A. Martin, and R. Di Pietro, “To docker or not to docker: A security perspective,” *IEEE Cloud Computing*, vol. 3, no. 5, pp. 54–62, 2016.
- [4] R. Shu, X. Gu, and W. Enck, “A study of security vulnerabilities on docker hub,” in *Proceedings of the 7th ACM on Conference on Data and Application Security and Privacy (CODASPY)*, 2017, pp. 269–280.
- [5] NIST National Vulnerability Database, “CVE-2022-0847: Dirty pipe — linux kernel privilege escalation,” <https://nvd.nist.gov/vuln/detail/CVE-2022-0847>, 2022.
- [6] —, “CVE-2019-5736: Docker runc container escape,” <https://nvd.nist.gov/vuln/detail/CVE-2019-5736>, 2019.
- [7] B. Gregg, “BPF performance tools: Linux system and application observability,” *Addison-Wesley Professional*, 2019, ISBN: 978-0-13-655482-1.
- [8] D. P. Kingma and M. Welling, “Auto-encoding variational bayes,” in *Proceedings of the 2nd International Conference on Learning Representations (ICLR)*, 2014.
- [9] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, 2008, pp. 413–422.

- [10] G. Duan, Y. Fu, M. Cai, H. Chen, and J. Sun, “DongTing: A large-scale dataset for anomaly detection of the Linux kernel,” *Journal of Systems and Software*, vol. 203, p. 111768, Sep. 2023.
- [11] M. Rostamipoor, S. Ghavamnia, and M. Polychronakis, “Confine: Fine-grained system call filtering for container attack surface reduction,” *Computers & Security*, vol. 132, p. 103338, Sep. 2023.
- [12] S. Kim, B. J. Kim, and D. H. Lee, “Prof-gen: Practical study on system call whitelist generation for container attack surface reduction,” in *Proceedings of the IEEE 14th International Conference on Cloud Computing (CLOUD)*, Sep. 2021, pp. 278–287.
- [13] J. Her, C. H. Jo, T. Park, and S. Lee, “KubeRosy: A dynamic system call filtering framework for containers,” *IEEE Access*, vol. 12, pp. 159 889–159 901, 2024.
- [14] S. Yang, B. B. Kang, and J. Nam, “Optimus: Association-based dynamic system call filtering for container attack surface reduction,” *Journal of Cloud Computing*, vol. 13, no. 1, p. 71, Mar. 2024.
- [15] J. Pei, J. Han, B. Mortazavi-Asl, J. Wang, H. Pinto, Q. Chen, U. Dayal, and M.-C. Hsu, “Mining sequential patterns by pattern-growth: The PrefixSpan approach,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 16, no. 11, pp. 1424–1440, 2004.
- [16] NIST National Vulnerability Database, “CVE-2021-4034: Polkit local privilege escalation,” <https://nvd.nist.gov/vuln/detail/CVE-2021-4034>, 2022.
- [17] S. Song, S. Suneja, M. V. Le, and B. Tak, “On the value of sequence-based system call filtering for container security,” in *Proceedings of the IEEE 16th International Conference on Cloud Computing (CLOUD)*, Jul. 2023, pp. 296–307.
- [18] NIST National Vulnerability Database, “CVE-2022-0185: Linux kernel heap overflow in filesystem context,” <https://nvd.nist.gov/vuln/detail/CVE-2022-0185>, 2022.
- [19] —, “CVE-2021-22555: Linux kernel netfilter heap out-of-bounds write,” <https://nvd.nist.gov/vuln/detail/CVE-2021-22555>, 2021.